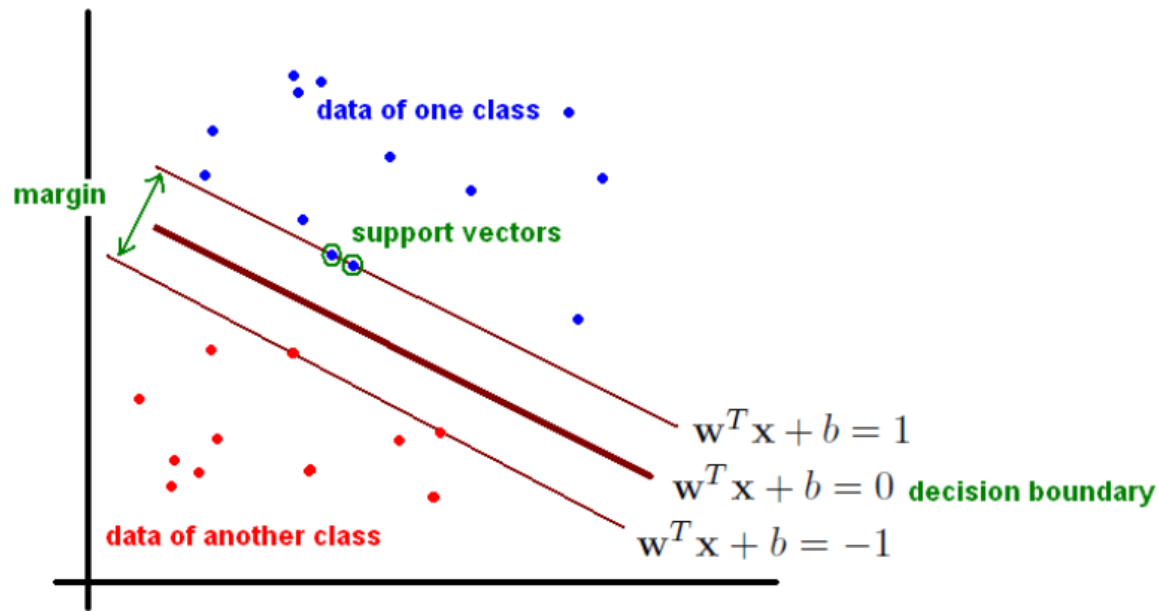


# Support Vector Machine



Goal: we want to find the hyperplane (i.e. decision boundary) linearly separating our classes. Our boundary will have equation:  $\mathbf{w}^T \mathbf{x} + b = 0$ .

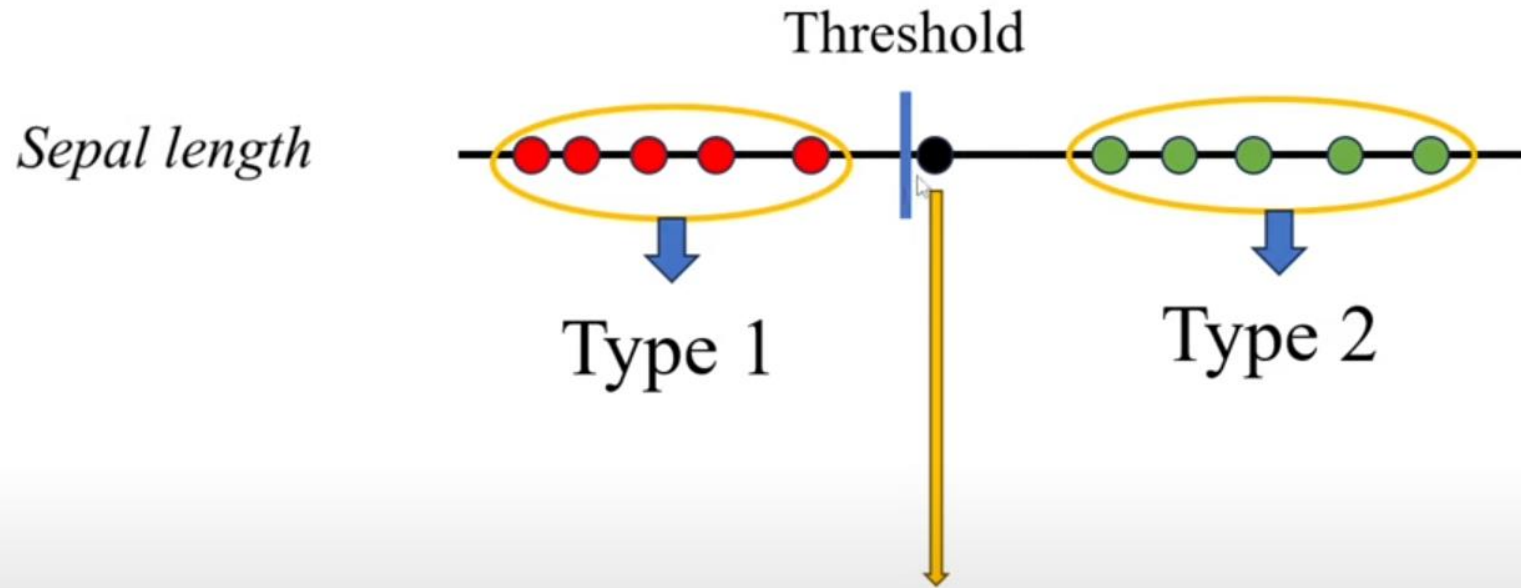
Anything above the decision boundary should have label 1.

i.e.,  $\mathbf{x}_i$  s.t.  $\mathbf{w}^T \mathbf{x}_i + b > 0$  will have corresponding  $y_i = 1$ .

Similarly, anything below the decision boundary should have label -1.

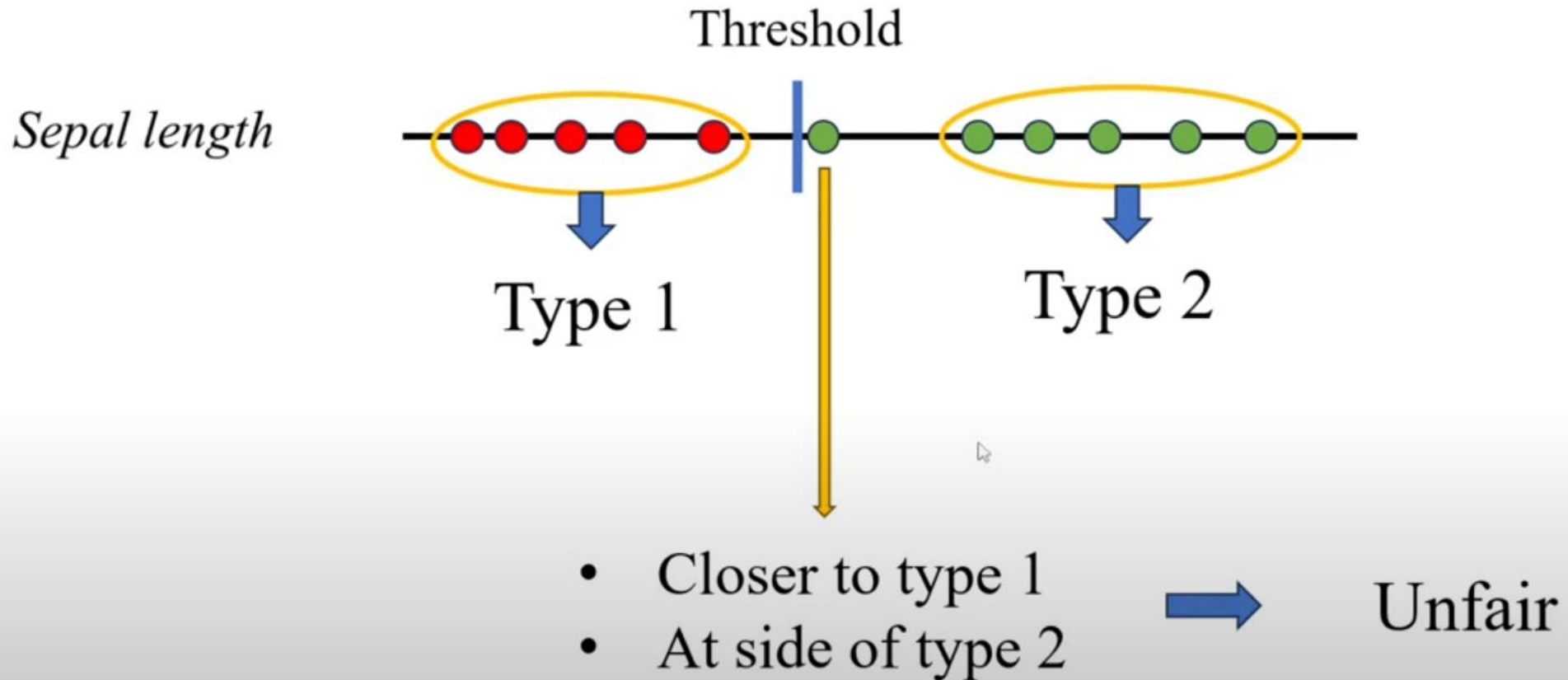
i.e.,  $\mathbf{x}_i$  s.t.  $\mathbf{w}^T \mathbf{x}_i + b < 0$  will have corresponding  $y_i = -1$ .

# Support Vector Machine

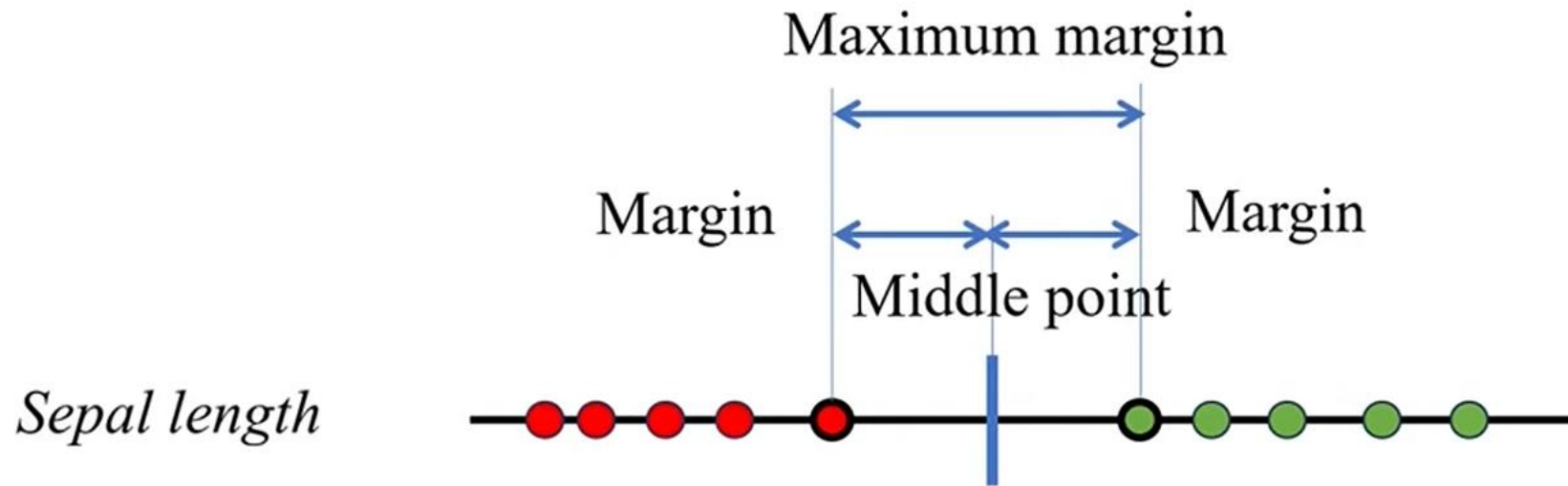


- Closer to type 1
- At side of type 2

# Support Vector Machine



# Support Vector Machine



**Maximum Margin Classification**

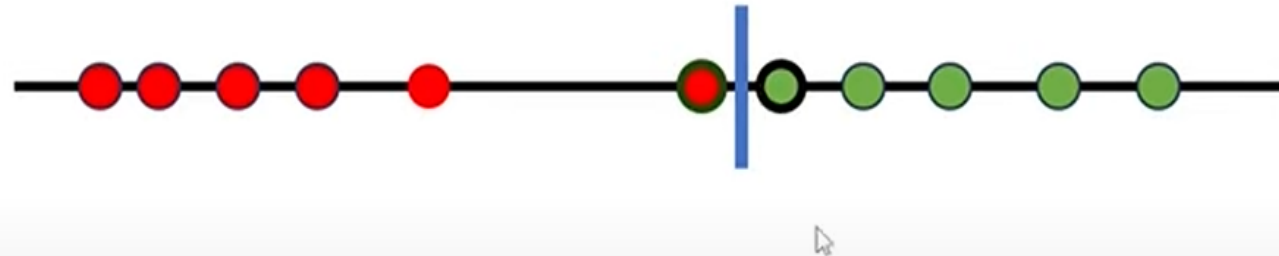
# Support Vector Machine

*Sepal length*



# Support Vector Machine

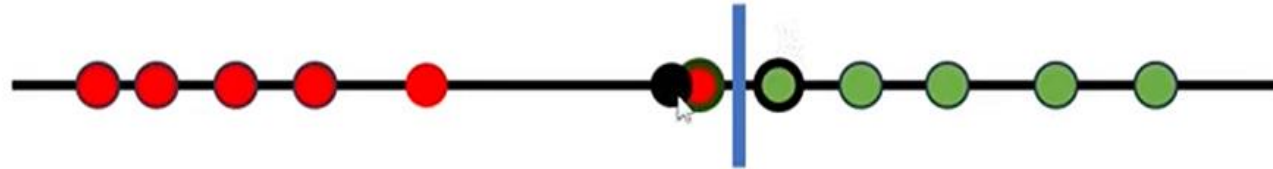
*Sepal length*



**Maximum Margin Classification**

# Support Vector Machine

*Sepal length*



**Maximum Margin Classification**

# Support Vector Machine

## Allowing misclassification

*Sepal length*

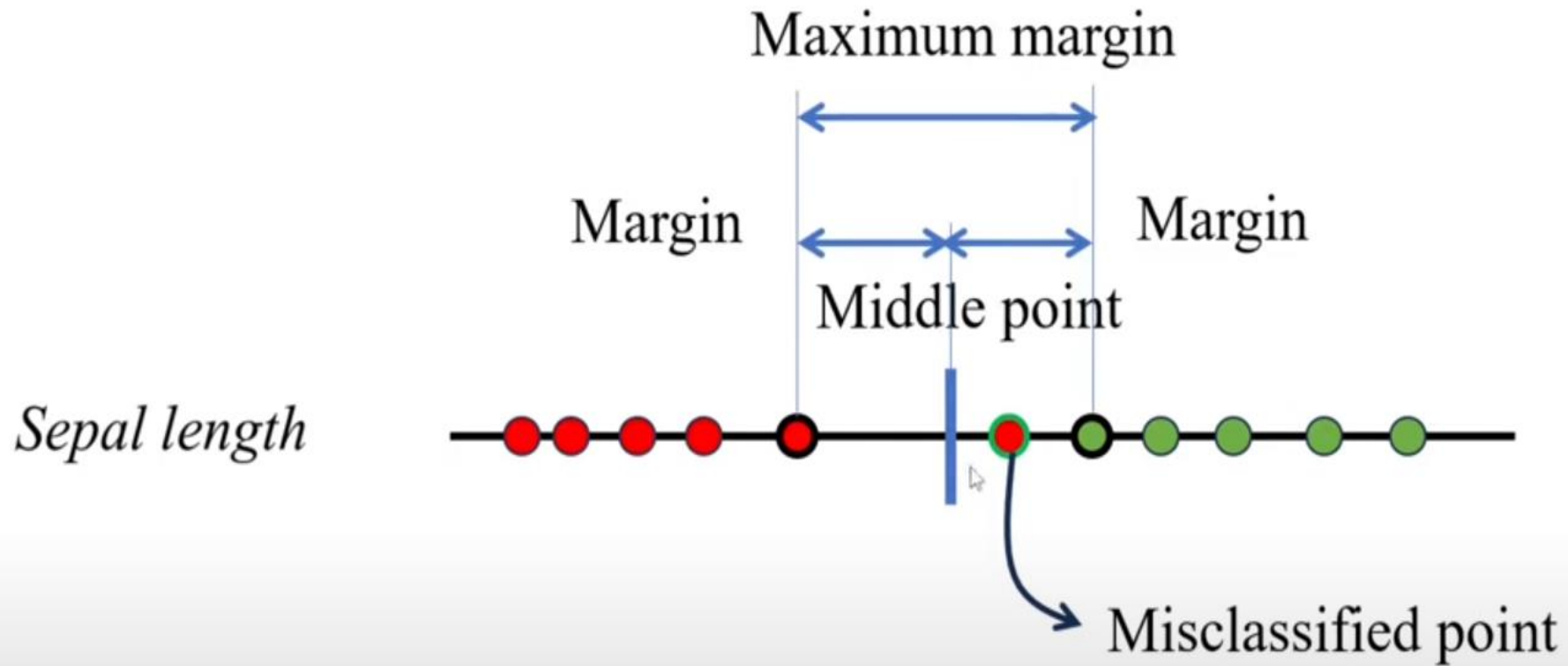


Misclassified point



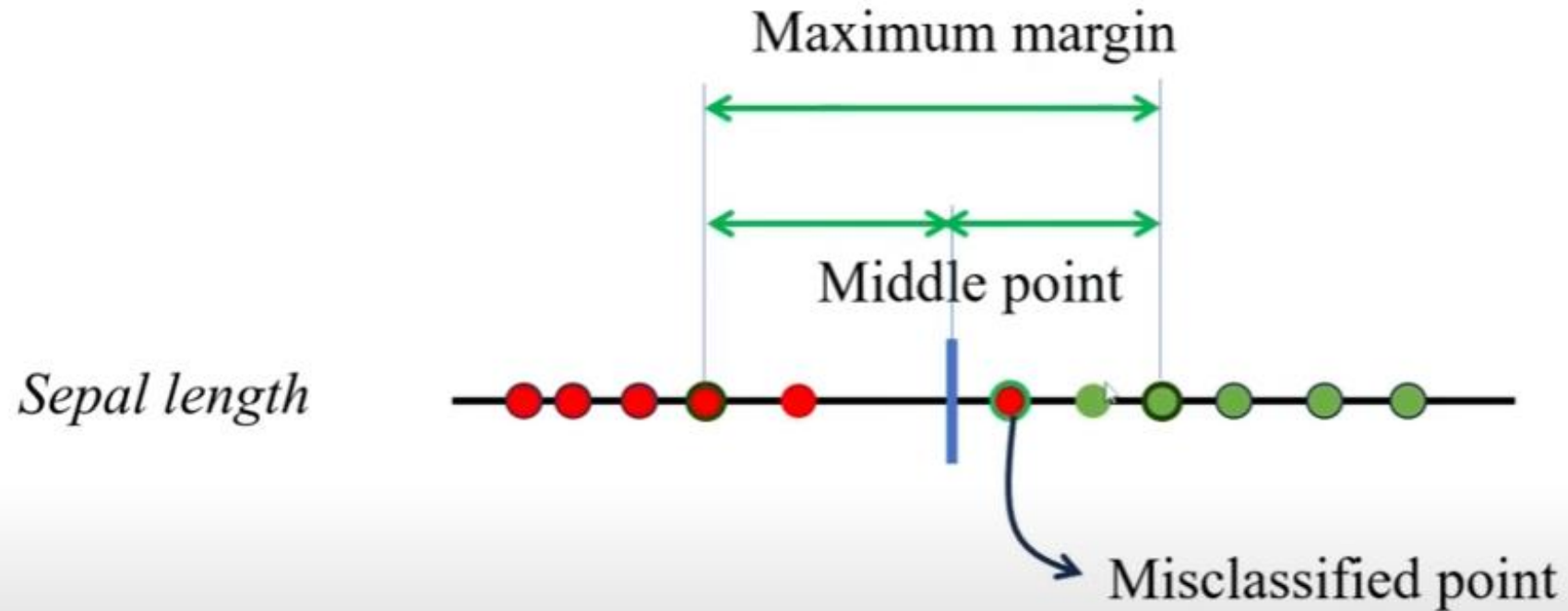
# Support Vector Machine

## Allowing misclassification



# Support Vector Machine

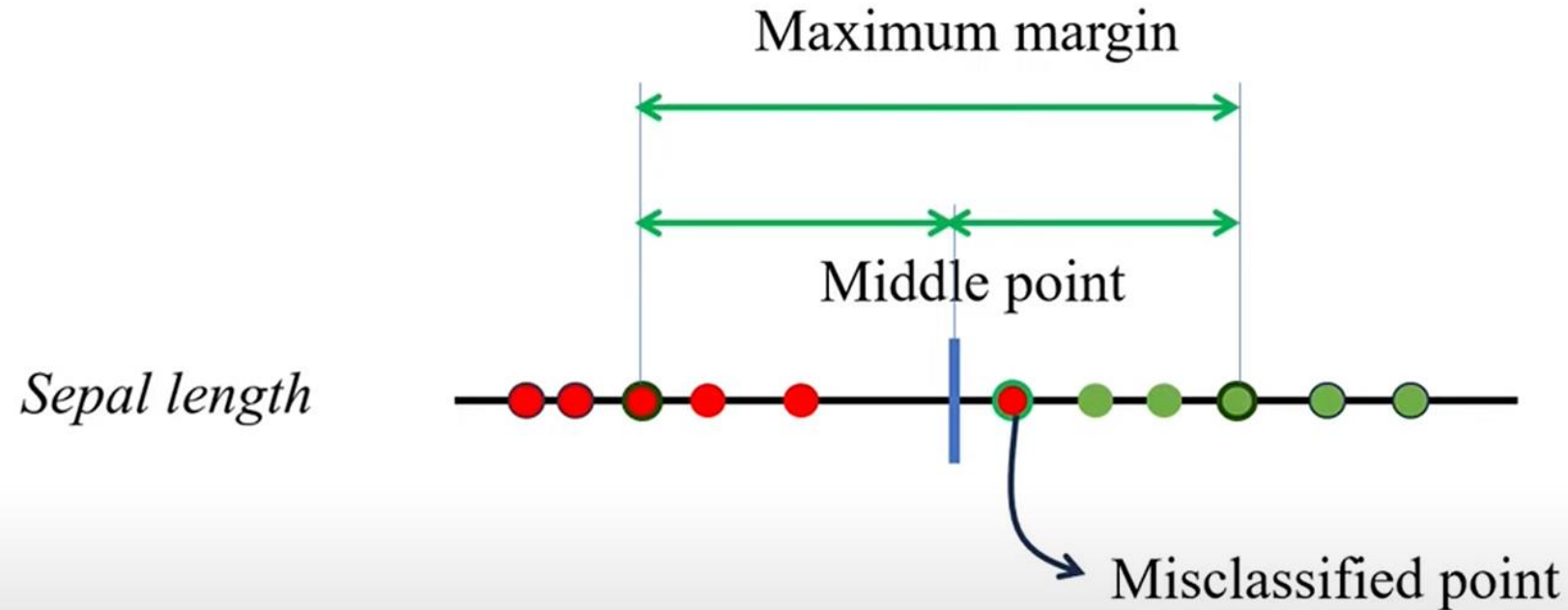
## Allowing misclassification



How to choose “”soft margin”???

# Support Vector Machine

## Allowing misclassification

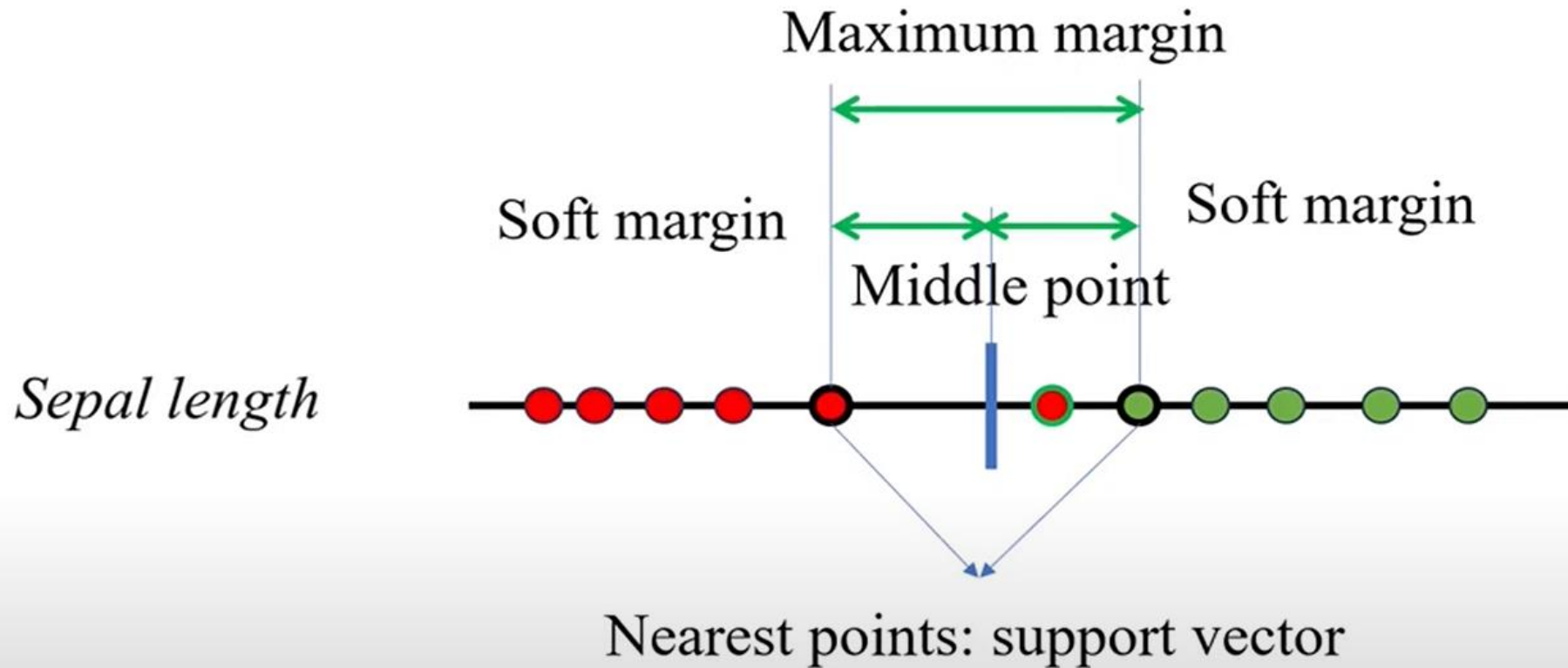


How to choose “”soft margin”???

Using Cross-validation to find the best soft margin

# Support Vector Machine

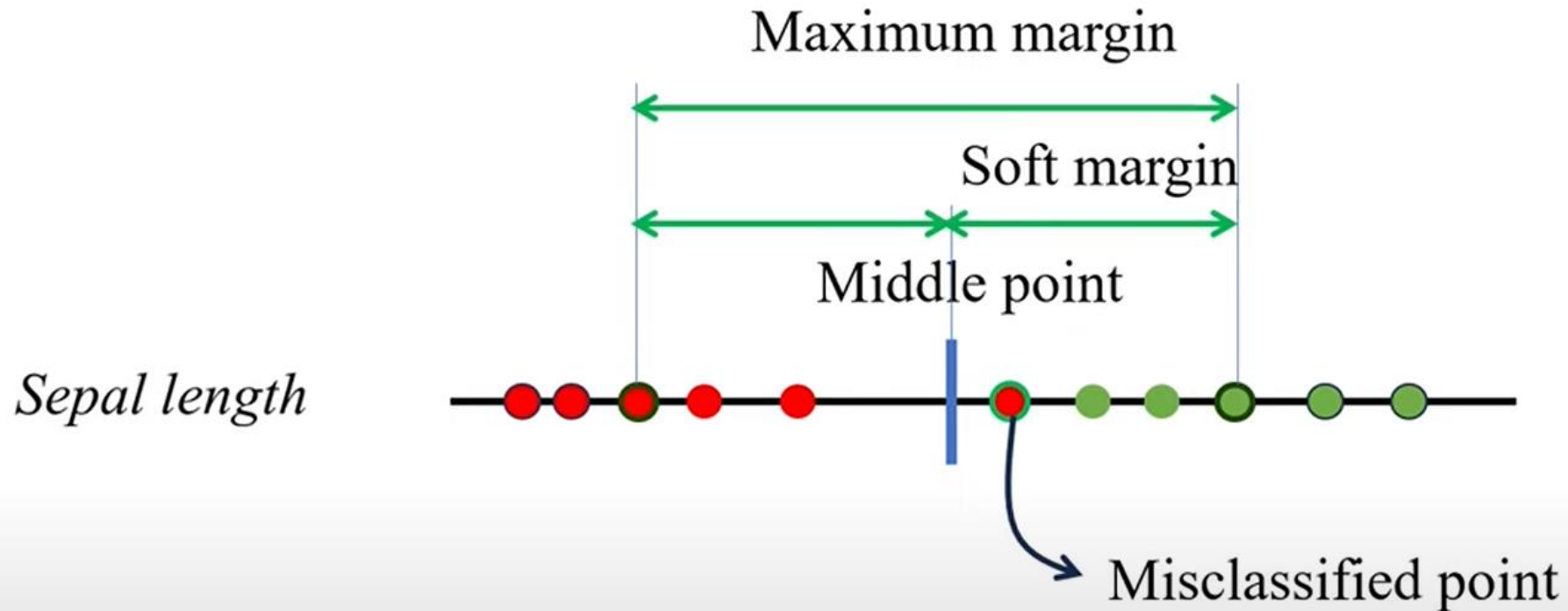
## Allowing misclassification



Support Vector Classification (SVC)

# Support Vector Machine

## Allowing misclassification



Choosing **soft margin** allows us to control **bias-variance tradeoff**

# Support Vector Machine

## Allowing misclassification



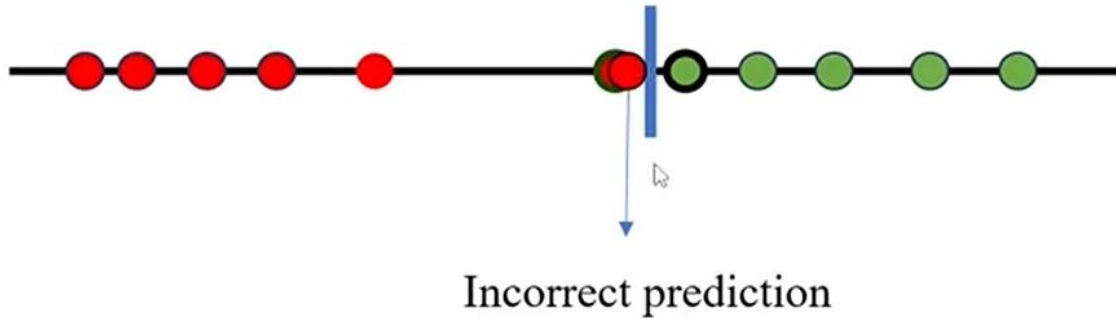
Less bias



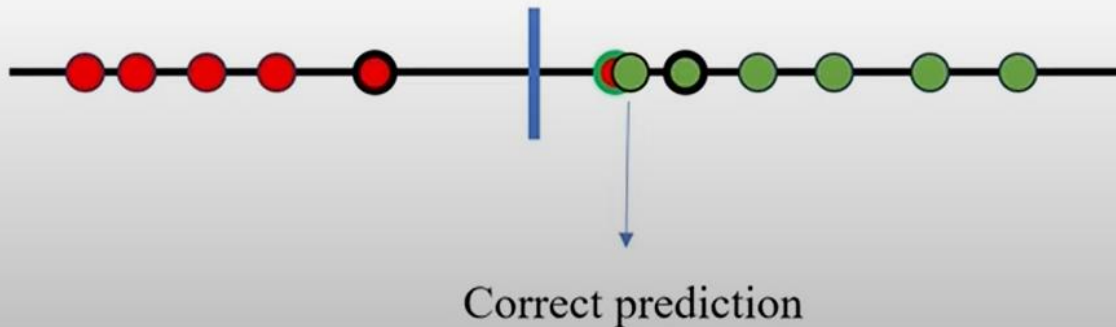
Higher bias

# Support Vector Machine

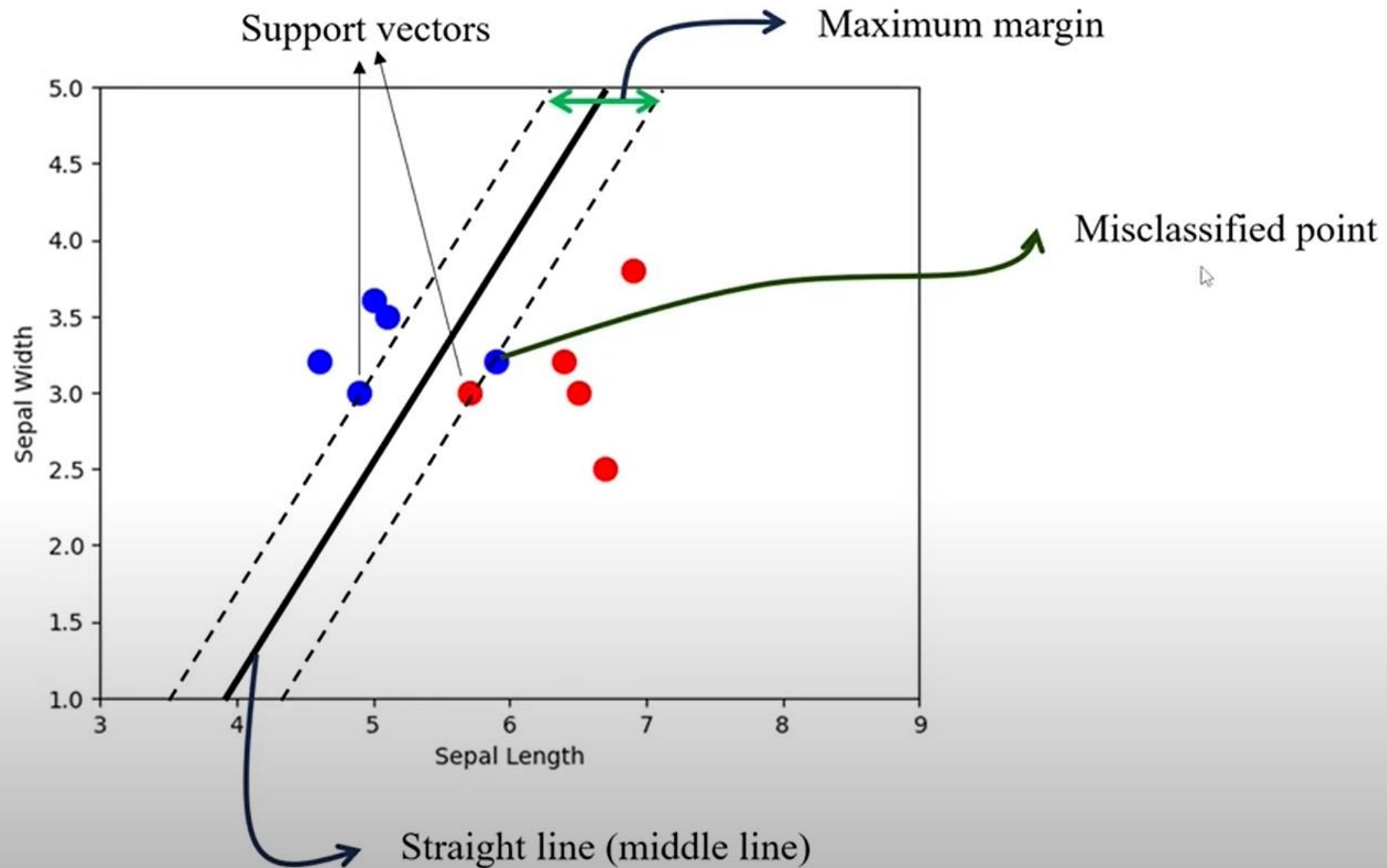
## Allowing misclassification



Less bias, higher variance

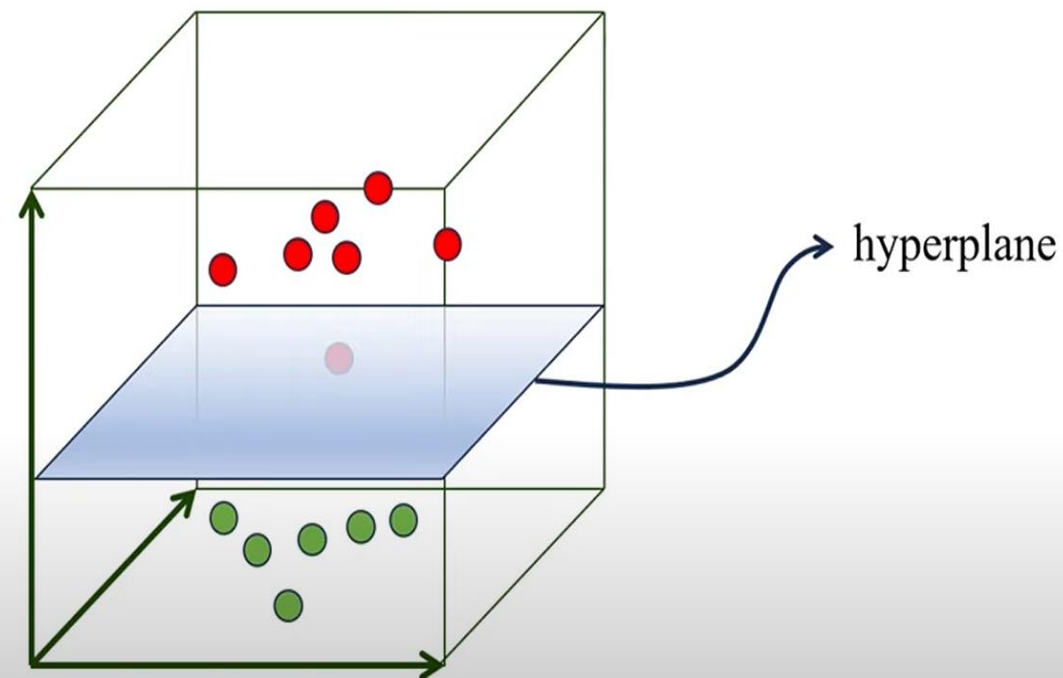
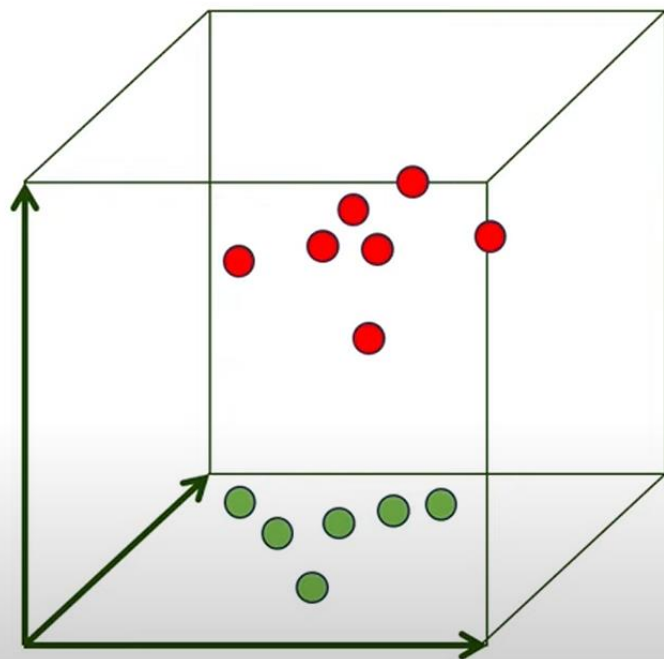


Higher bias, low variance



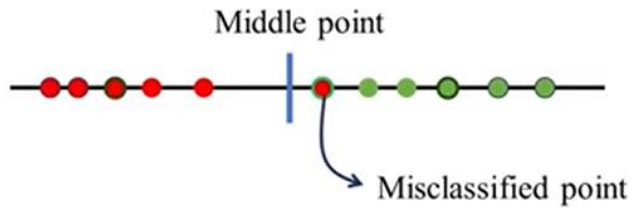


# Support Vector Machine



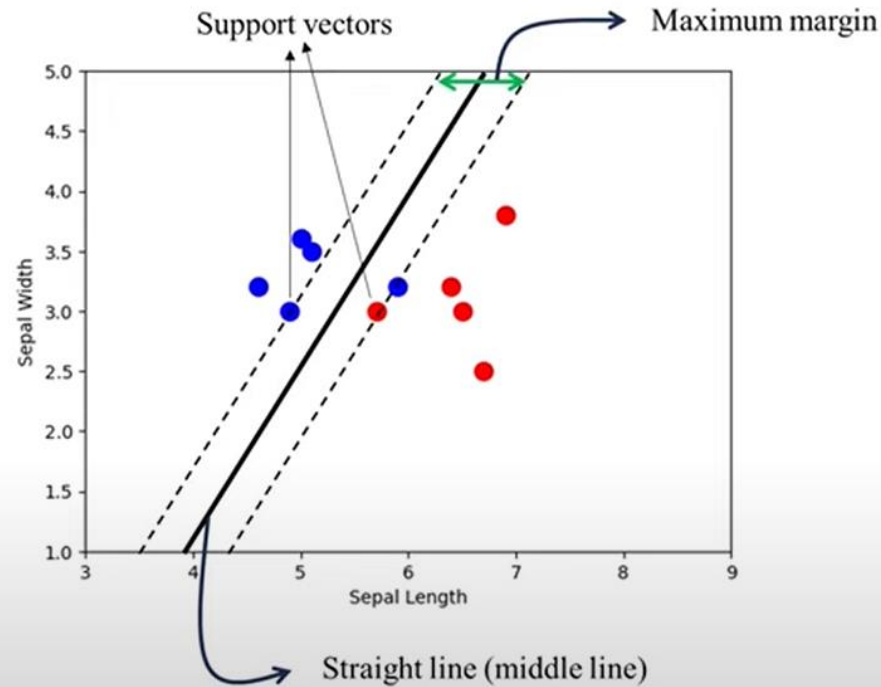
# Support Vector Machine

1-dimensional data



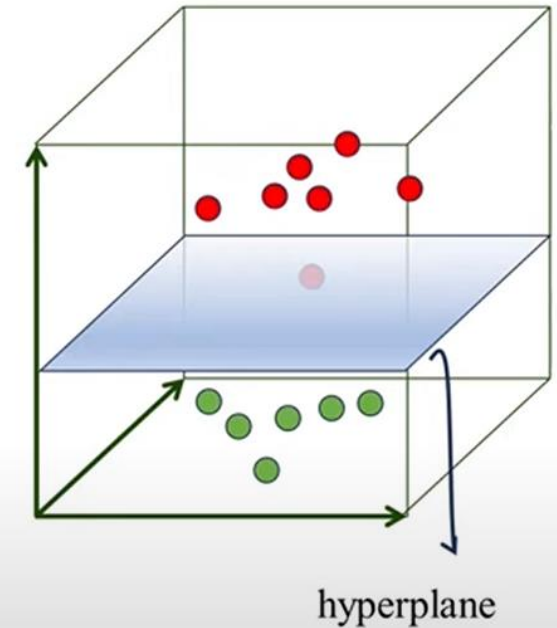
SVC is a single point

2-dimensional data



SVC is a 1D line

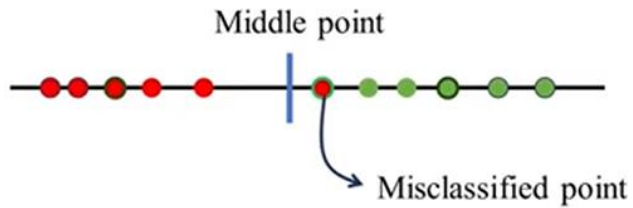
3-dimensional data



SVC is a 2D plane

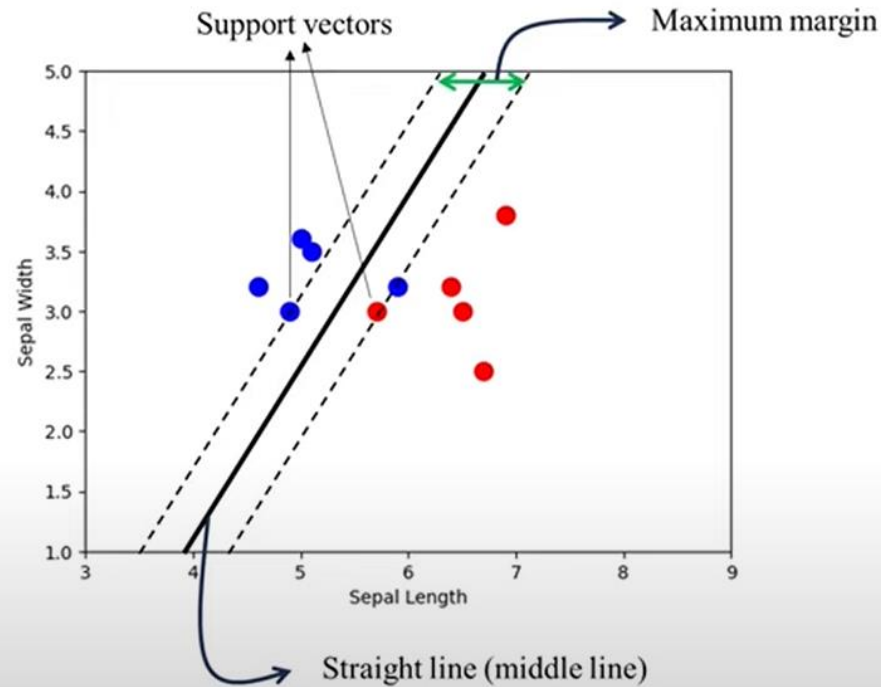
# Support Vector Machine

1-dimensional data



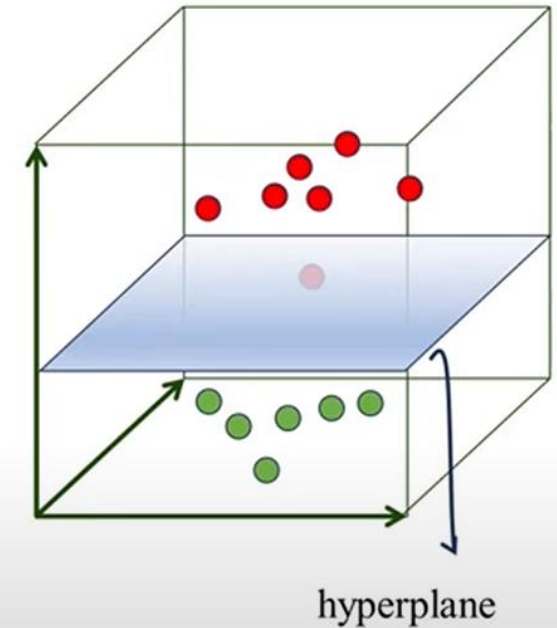
SVC is a single point

2-dimensional data



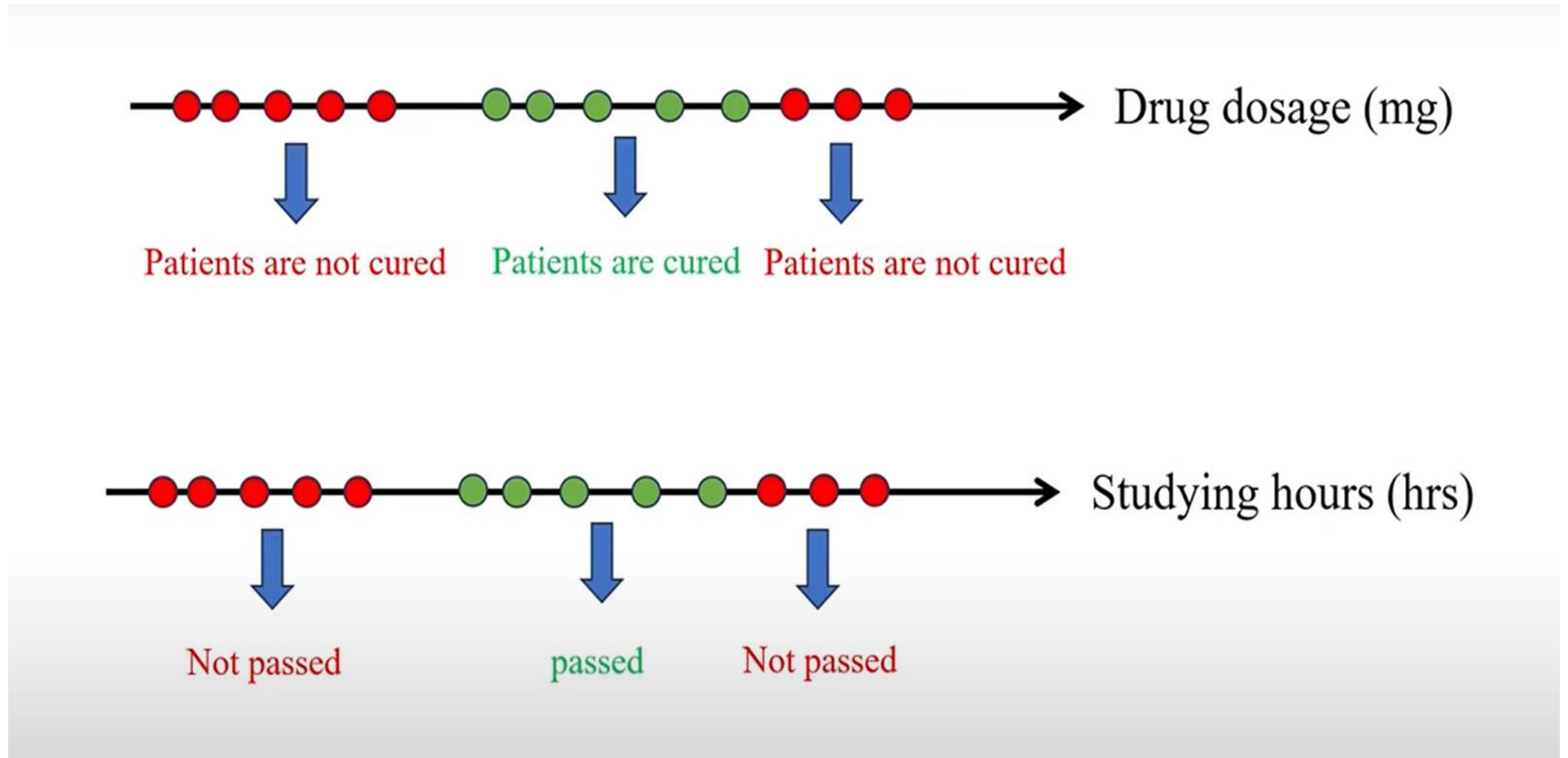
SVC is a 1D line

3-dimensional data



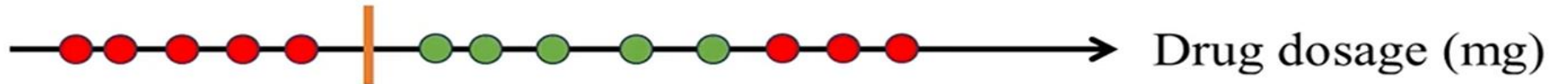
SVC is a 2D plane

# Support Vector Classification



# Support Vector Machine

For non-linear problem



Linear SVC can not be applied for

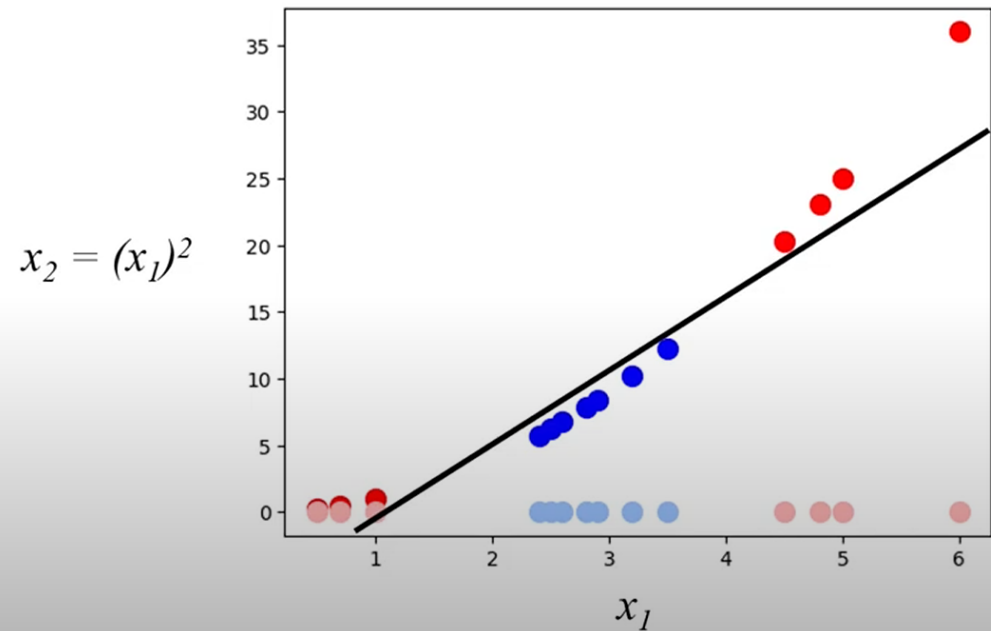
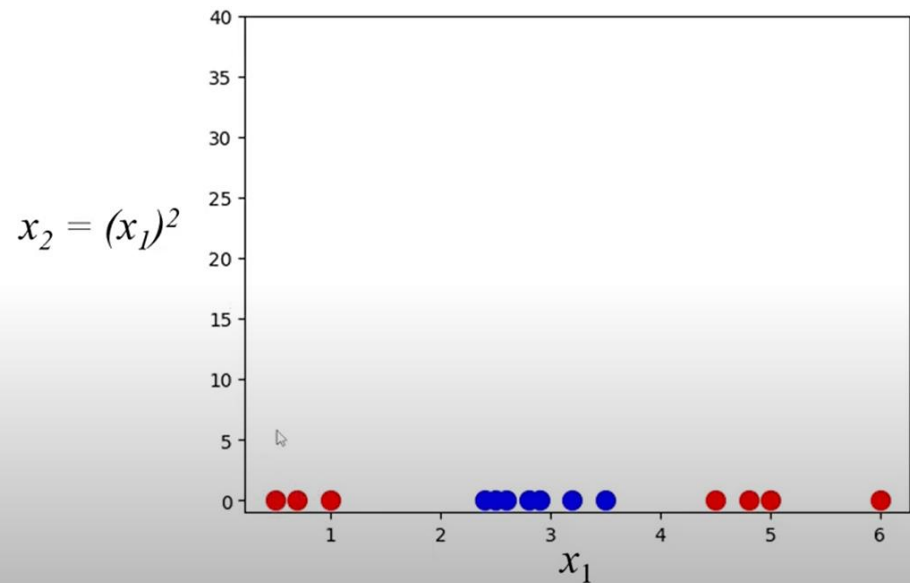


using Kernel trick

The kernel trick is a technique to transform data from a lower-dimensional space into a higher-dimensional space without explicitly computing the transformation.

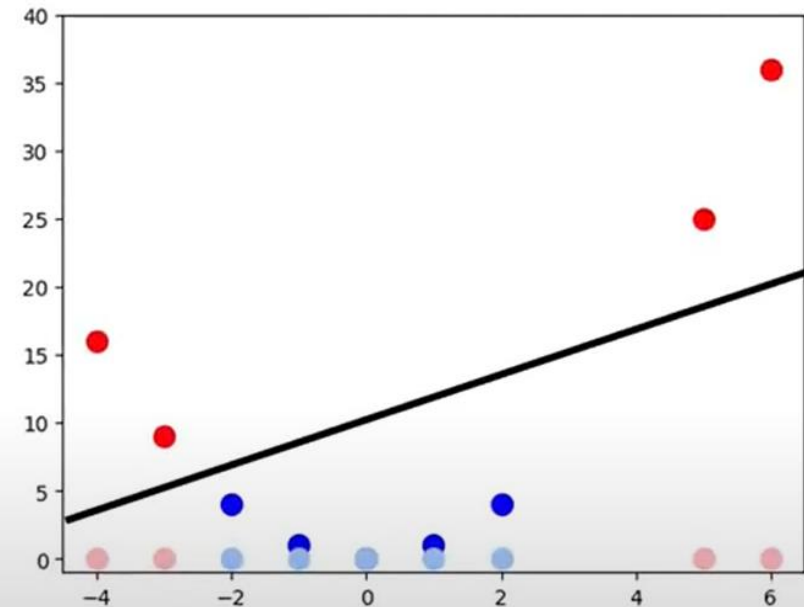
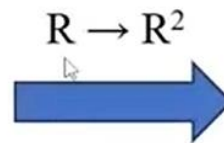
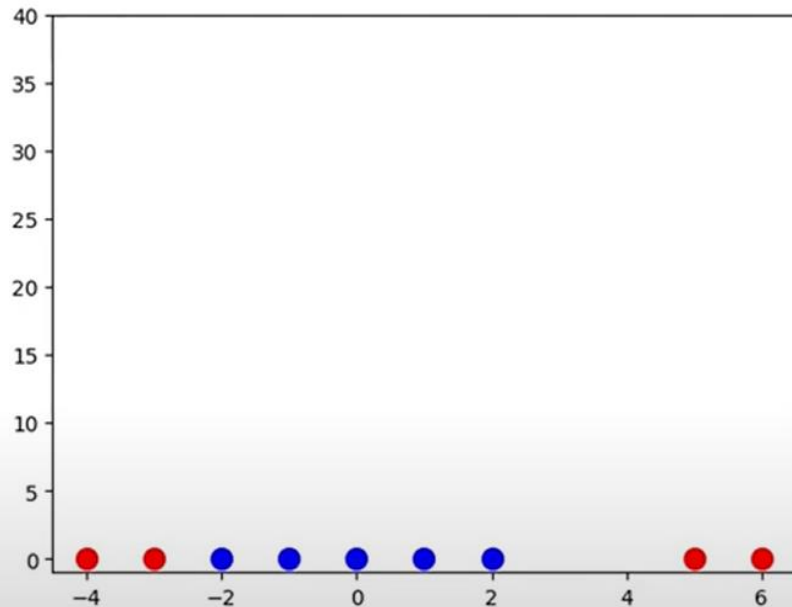
# Support Vector Machine

The kernel trick is a technique to transform data from a **lower-dimensional space** into a **higher-dimensional space** without explicitly computing the transformation.



# Support Vector Machine

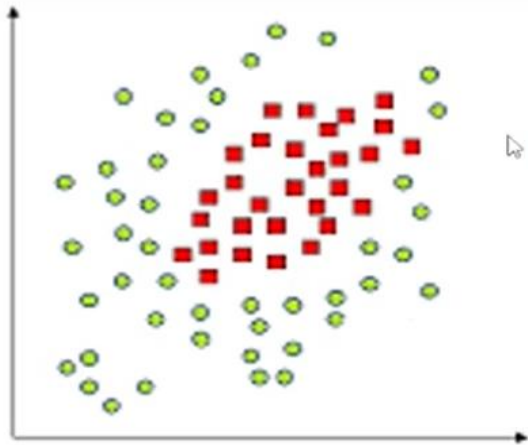
The kernel trick is a technique to transform data from a **lower-dimensional space** into a **higher-dimensional space** without explicitly computing the transformation.



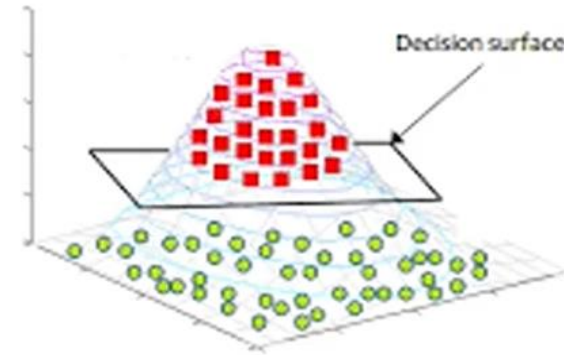


# Support Vector Machine

The kernel trick is a technique to transform data from a **lower-dimensional space** into a **higher-dimensional space** without explicitly computing the transformation.



Feature Mapping



Figures obtained from Medium

Original input  $x$

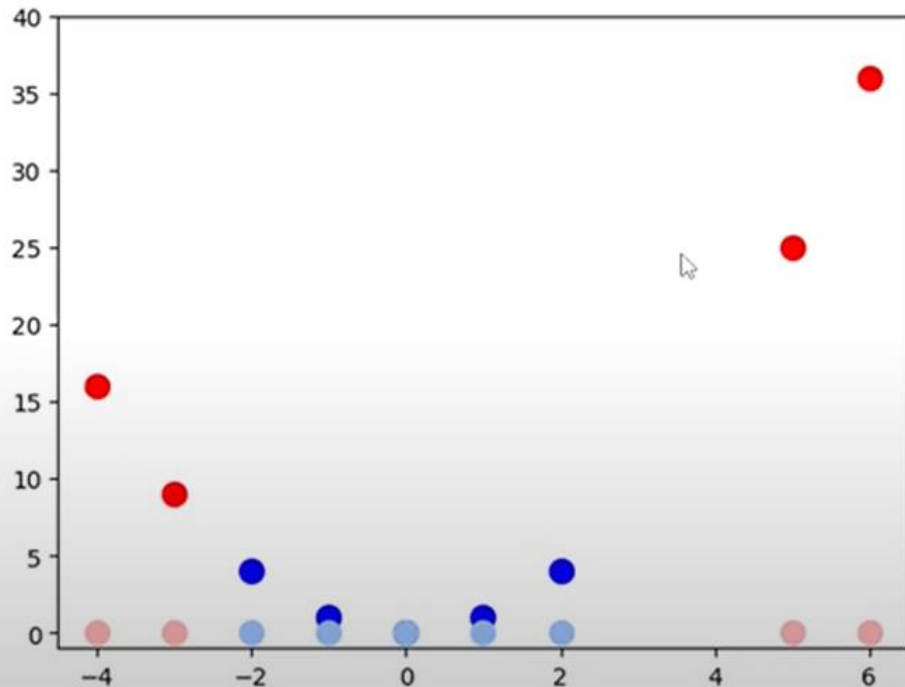


Feature mapping is a function  $\Phi(x)$

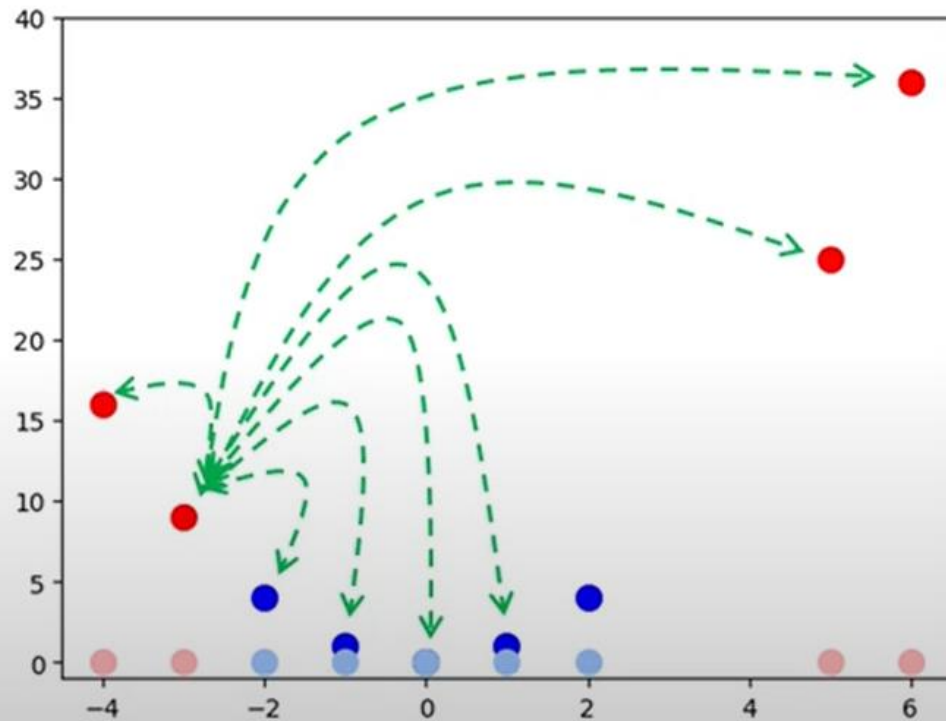


# Support Vector Machine

To optimize model and find hyperplane, SVM algorithm needs to calculate relationship between all pairs of data points using dot products  $\phi(x_i)^T \phi(x_j)$



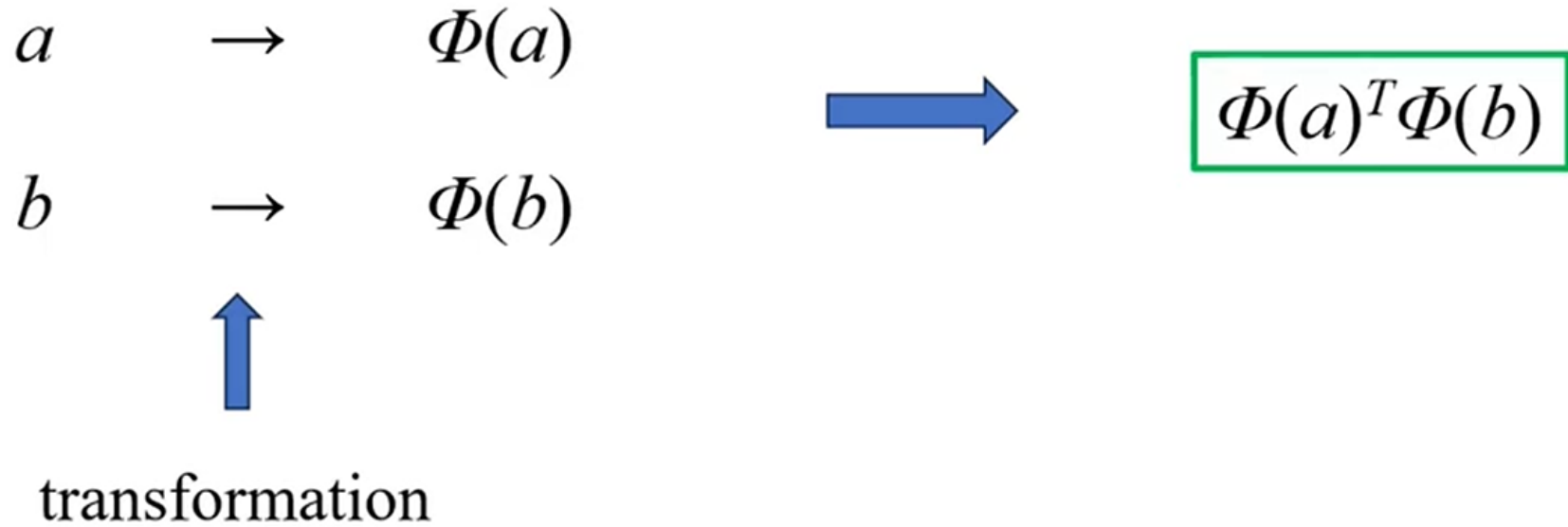
To optimize model and find hyperplane, SVM algorithm needs to calculate relationship between all pairs of data points using dot products:  $\phi(x^{(i)})^T \phi(x^{(j)})$



In very high- dimension space, it is very difficult to calculate  $\phi(x^{(i)})^T \phi(x^{(j)})$



**Kernel function is helpful to solve problem**



Kernel function (K)

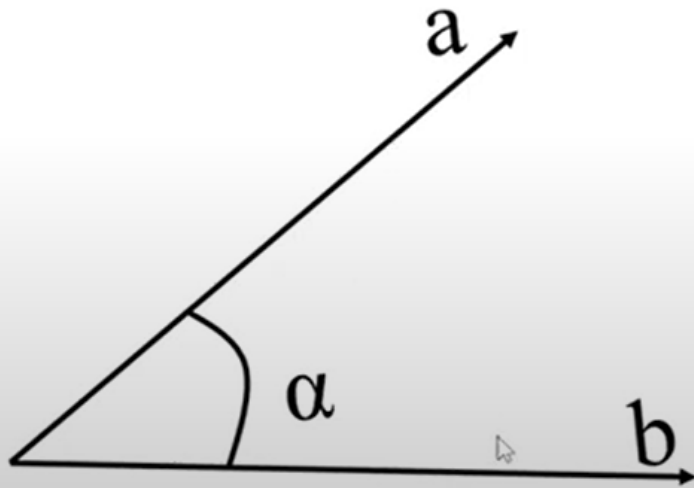
$$K(a,b) = \Phi(a)^T \Phi(b)$$

Try to find  $\Phi(a)^T \Phi(b)$  without transforming the space.

If you have two vectors:  $a = (a_1, a_2)$  and  $b = (b_1, b_2)$

Dot product of a and b is:

$$a.b = a^T b = a_1 b_1 + a_2 b_2 = \sum_i a_i b_i$$

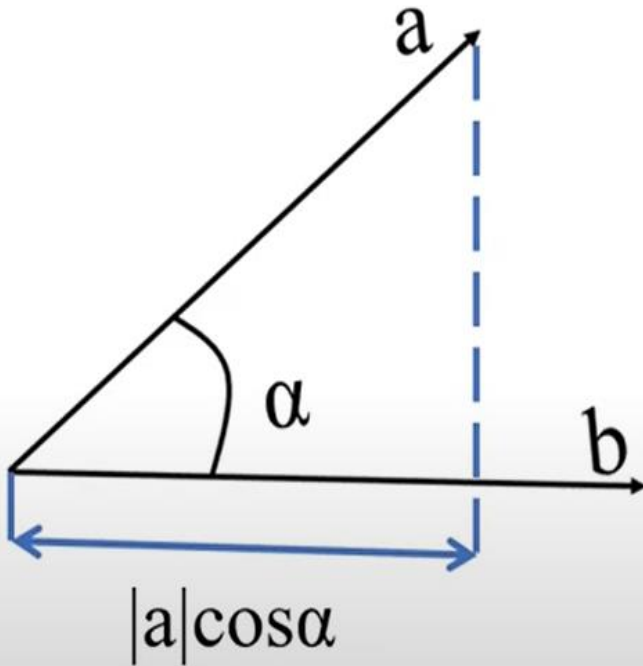


$$a.b = \|a\|.\|b\|.\cos\alpha$$

- $a.b = \|a\|.\|b\|.\cos\alpha = 0$  when  $\alpha = 90$
- $a.b = \|a\|.\|b\|$  when  $\alpha = 0$
- $a.a = \|a\|^2 \rightarrow \|a\| = \sqrt{a.a}$

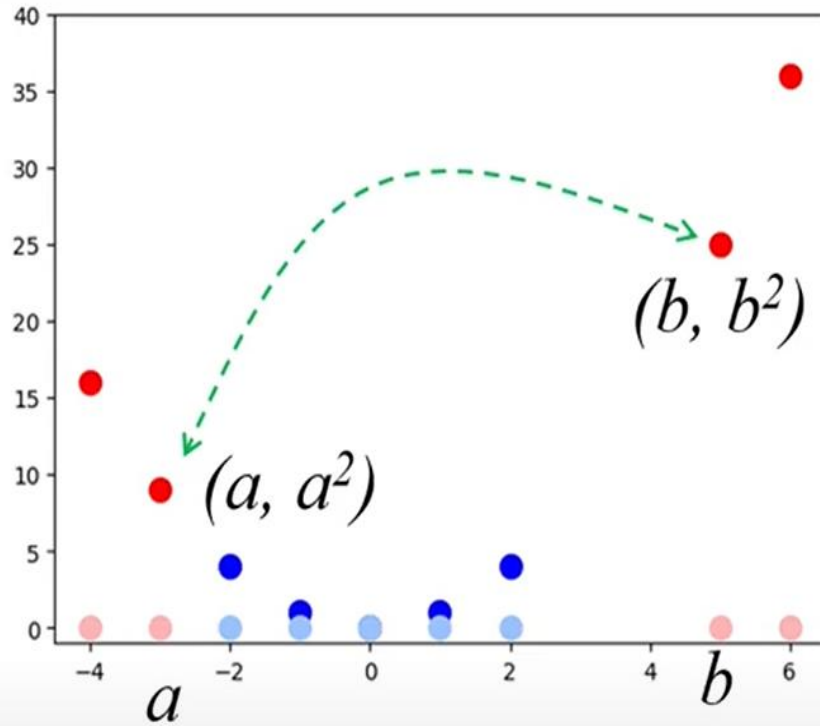
# Support Vector Machine

Scalar projection of a Euclidean vector  $a$  in the direction of a Euclidean vector  $b$ :



$$a_b = \|a\| \cdot \cos\alpha$$

# Support Vector Machine



$$\begin{aligned}\Phi(a)^T \Phi(b) &= (a, a^2)^T (b, b^2) \\ &= ab + a^2 b^2\end{aligned}$$

## Polynomial Kernel

$$K(a, b) = (a \cdot b + c)^d$$

d: degree of polynomial, c is user-defined parameter

Case:  $c = 1/2$  and  $d = 2$

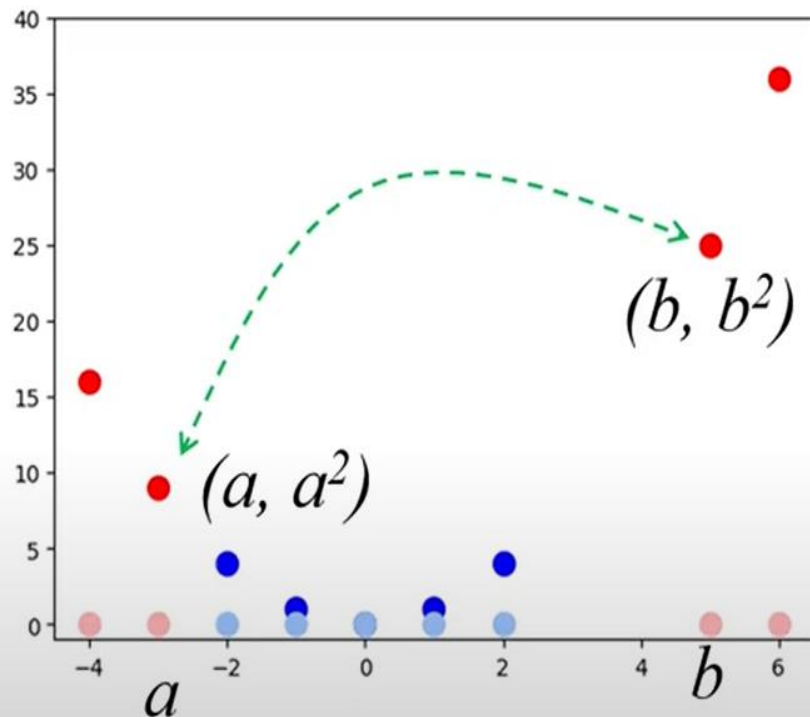
$$K(a, b) = (a \cdot b + 1/2)^2$$

$$\begin{aligned}K(a, b) &= (ab + 1/2)(ab + 1/2) \\ &= a^2 b^2 + 1/2 ab + 1/2 ab + 1/4 \\ &= a^2 b^2 + ab + 1/4 \\ &= ab + a^2 b^2 + (1/2)^2 \\ &= (a, a^2, 1/2)^T (b, b^2, 1/2)\end{aligned}$$

# Support Vector Machine

## Polynomial Kernel

- Using features at original data (low dimension)



$$K(a, b) = (a \cdot b + c)^d$$

$$K(a, b) = (a, a^2, 1/2)^T (b, b^2, 1/2)$$

- Using features transformed by basic function (high dimension)

$$\Phi(a)^T \Phi(b) = (a, a^2)^T (b, b^2)$$

$$K(a, b) = \Phi(a)^T \Phi(b)$$



# Support Vector Machine

The kernel trick is a technique to transform data from a **lower-dimensional space** into a **higher-dimensional space** without explicitly computing the transformation.

## Polynomial Kernel

$$K(x, y) = (x^T y + c)^d$$

## Radical Basis Function (RBF) Kernel

$$K(x, y) = e^{(-\gamma \|x - y\|^2)}$$

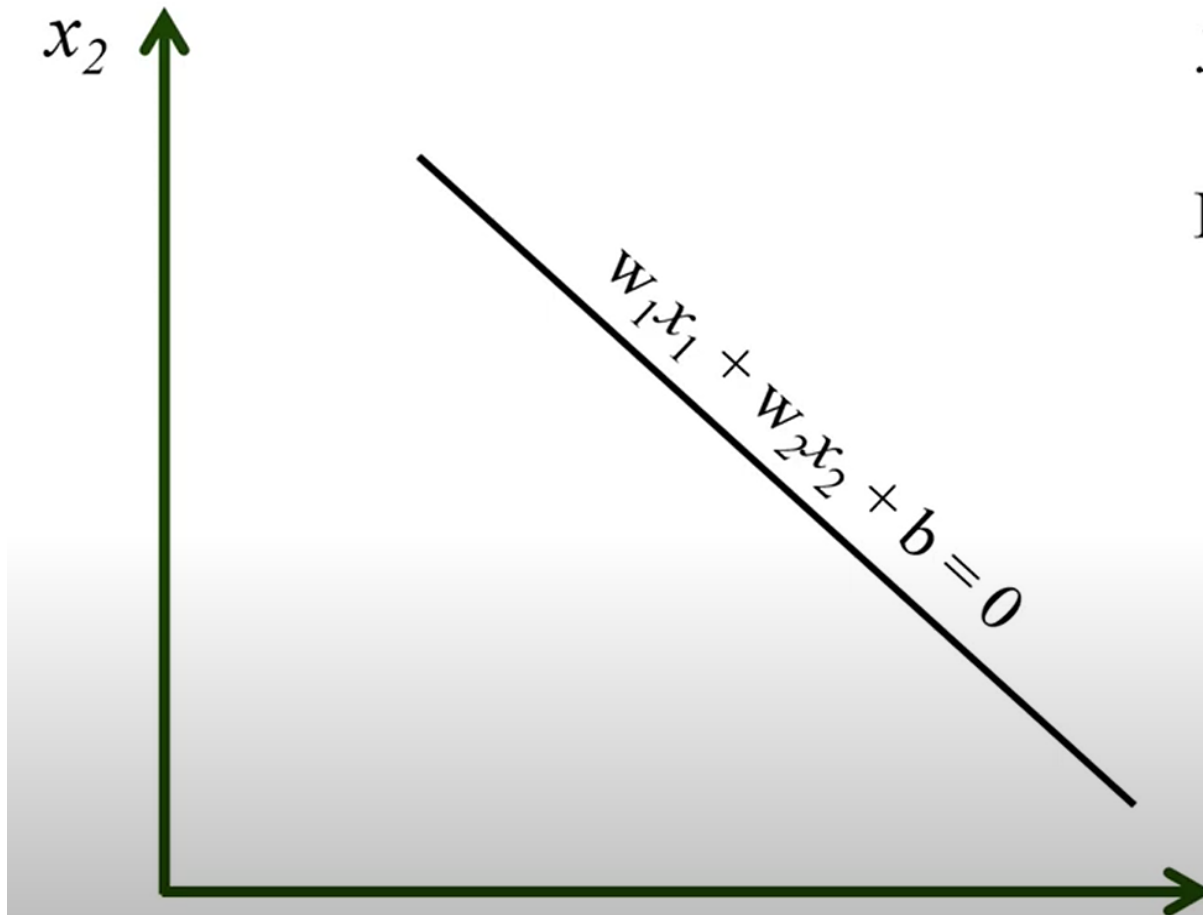
## Sigmoid Kernel

$$K(x, y) = \tanh(\alpha x^T y + c)$$

$\tanh$ : hyperbolic tangent



# Support Vector Machine



$$y = ax + b \quad \longrightarrow \quad ax - y + b = 0$$

Linear equation

$$w_1x_1 + w_2x_2 + b = 0$$

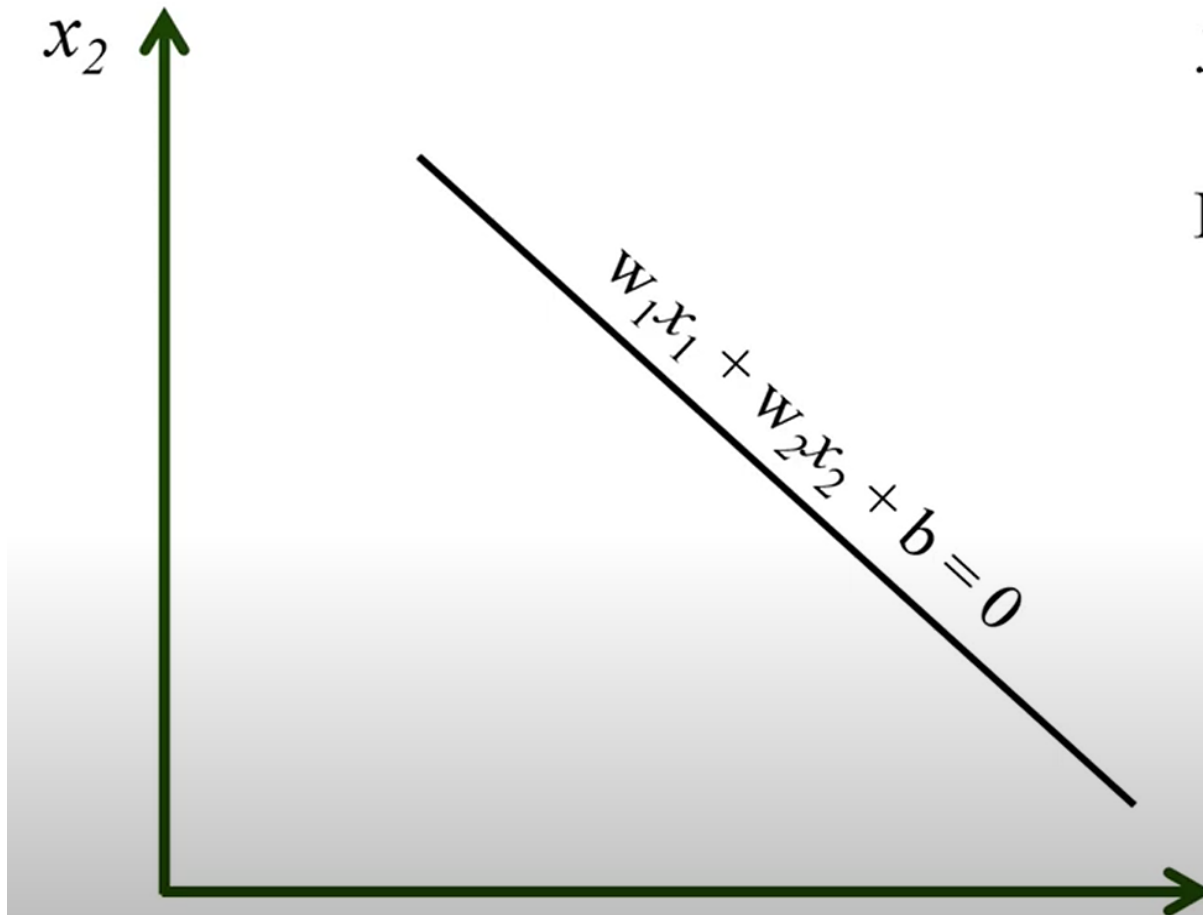
$-w_1/w_2$  is slope of line

Equation

$$\mathbf{w}^T \mathbf{x} + b = 0$$

$$\mathbf{w} = (w_1, w_2)$$
$$\mathbf{x} = (x_1, x_2)$$

# Support Vector Machine



$$y = ax + b \quad \longrightarrow \quad ax - y + b = 0$$

Linear equation

$$w_1x_1 + w_2x_2 + b = 0$$

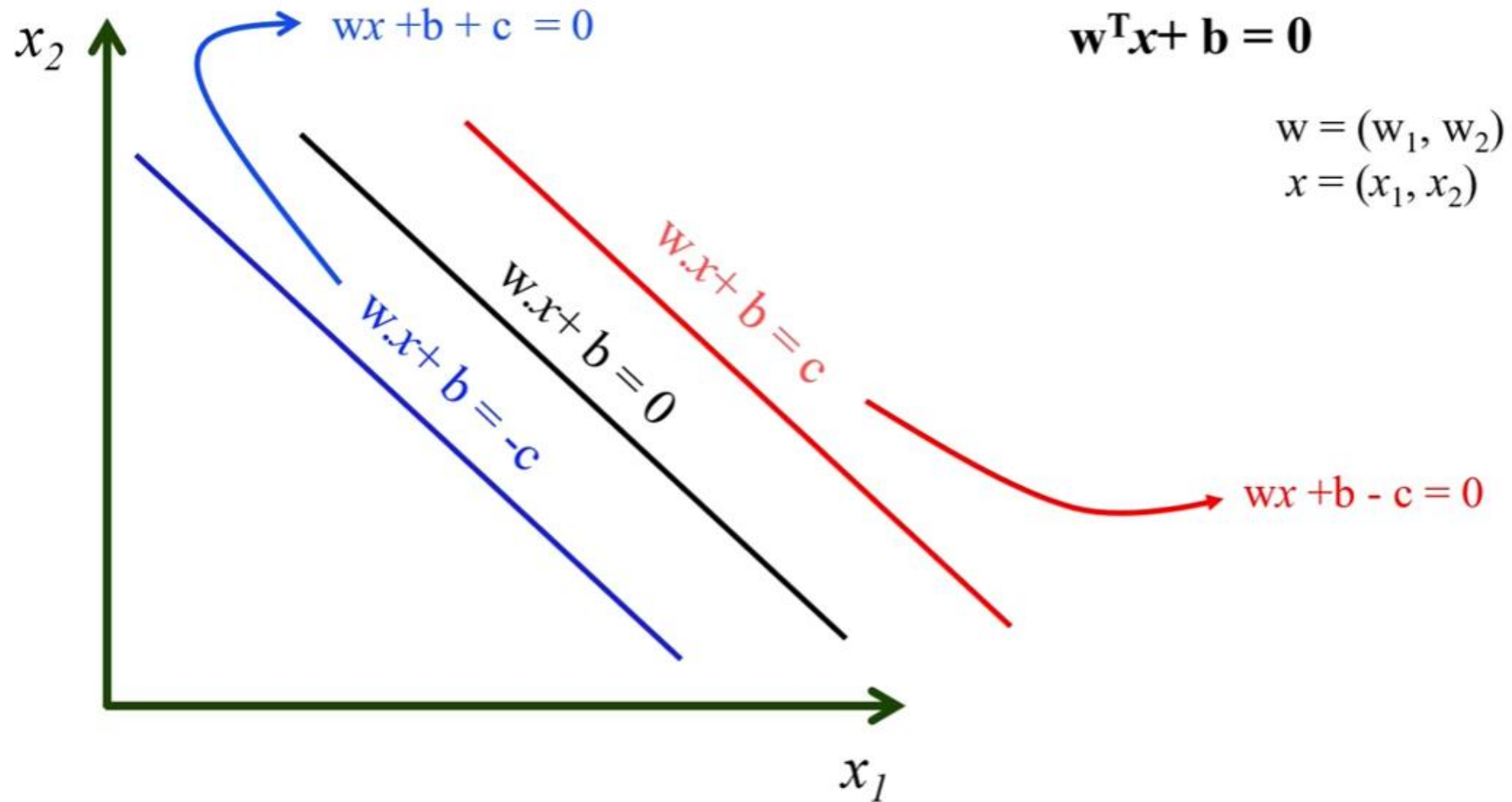
$-w_1/w_2$  is slope of line

Equation

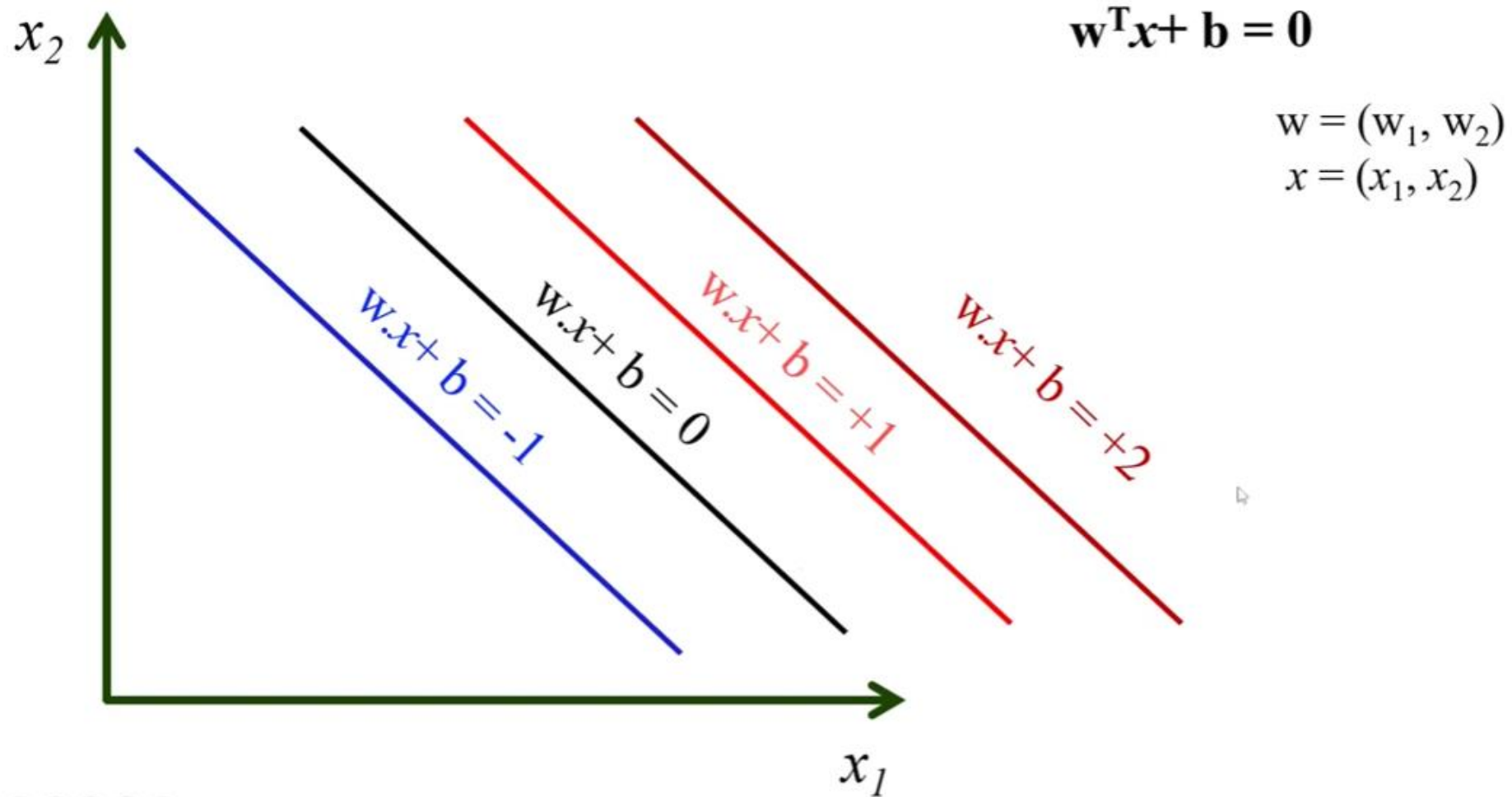
$$\mathbf{w}^T \mathbf{x} + b = 0$$

$$\mathbf{w} = (w_1, w_2)$$
$$\mathbf{x} = (x_1, x_2)$$

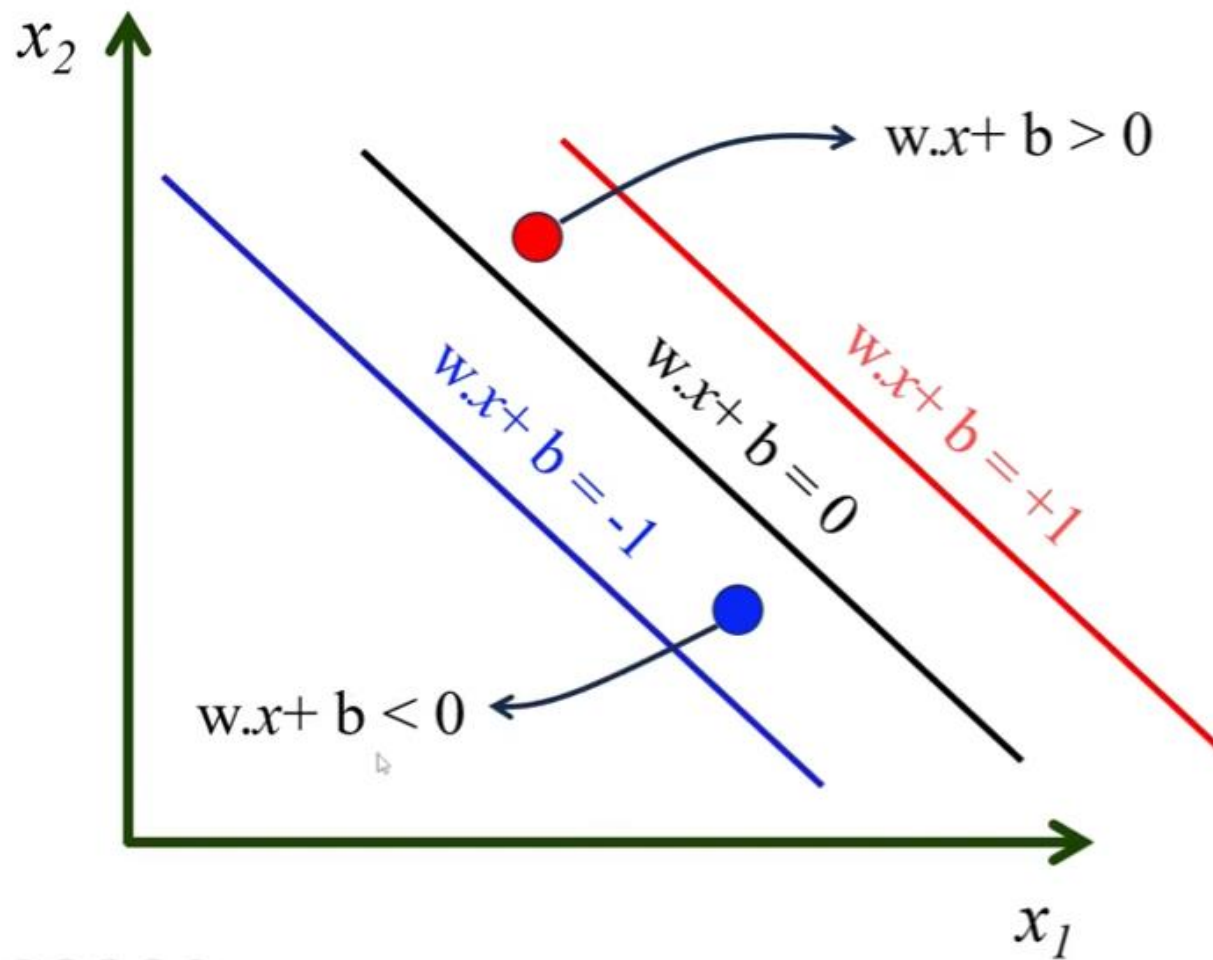
# Support Vector Machine



# Support Vector Machine



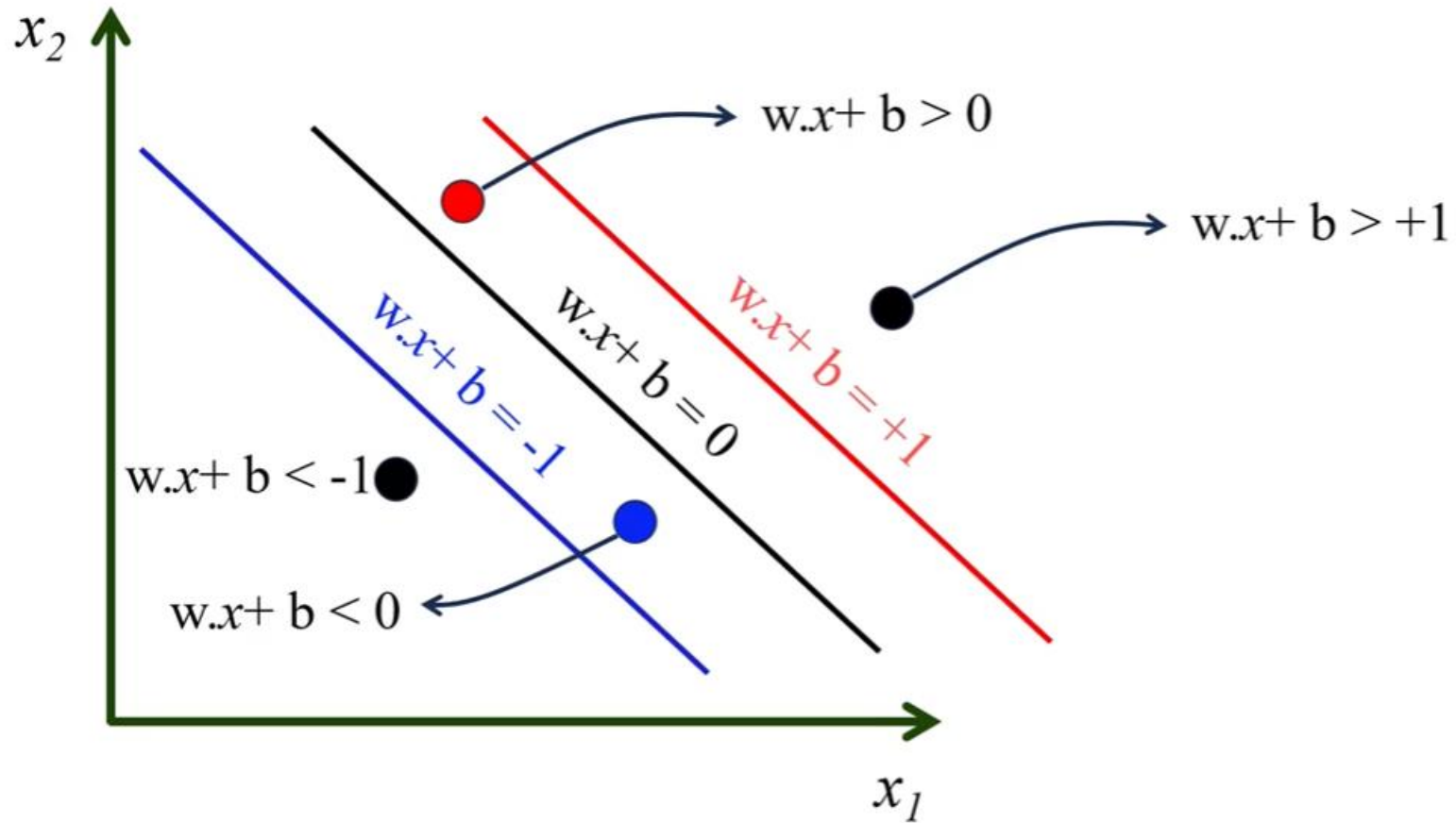
# Support Vector Machine



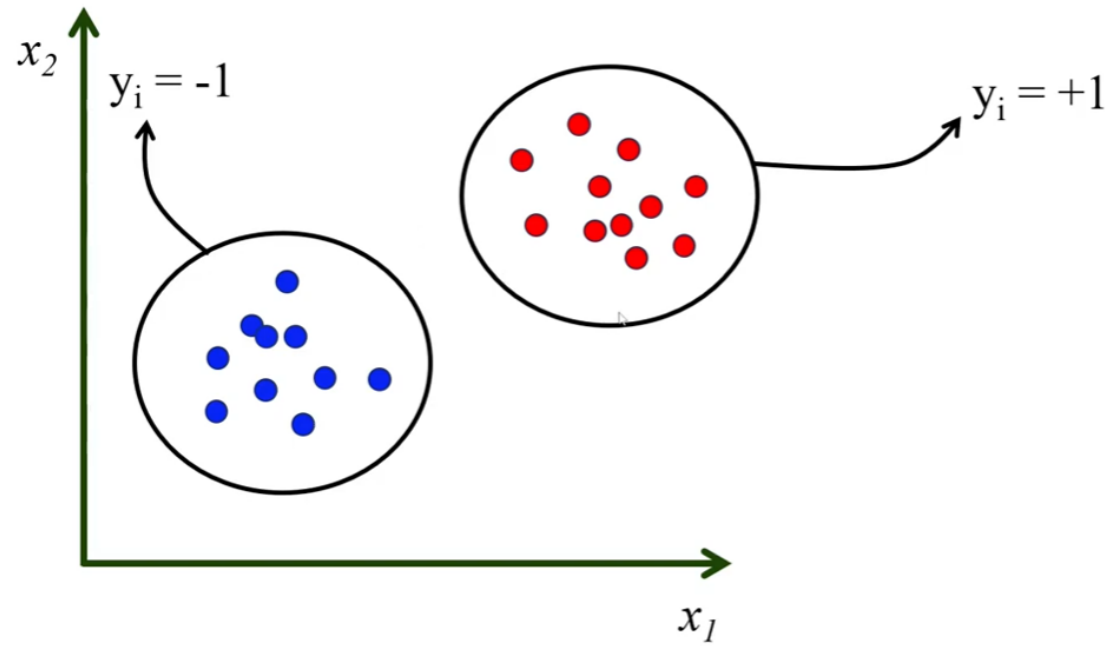
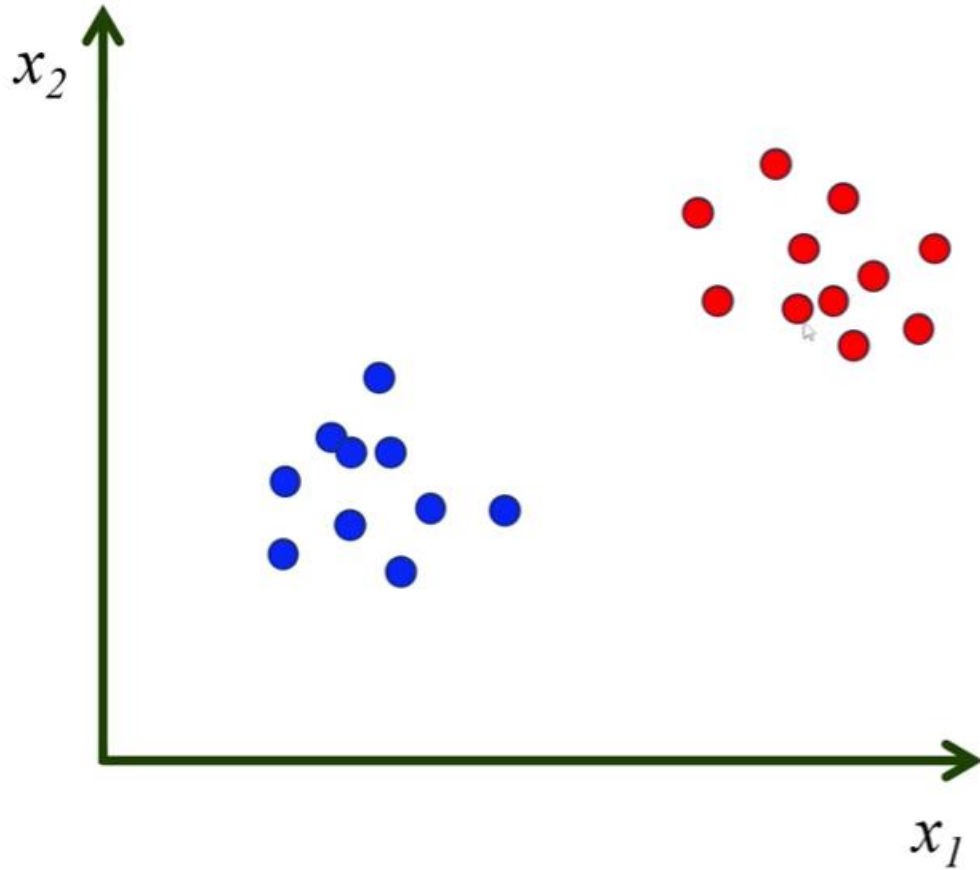
$$w^T x + b = 0$$

$$w = (w_1, w_2)$$
$$x = (x_1, x_2)$$

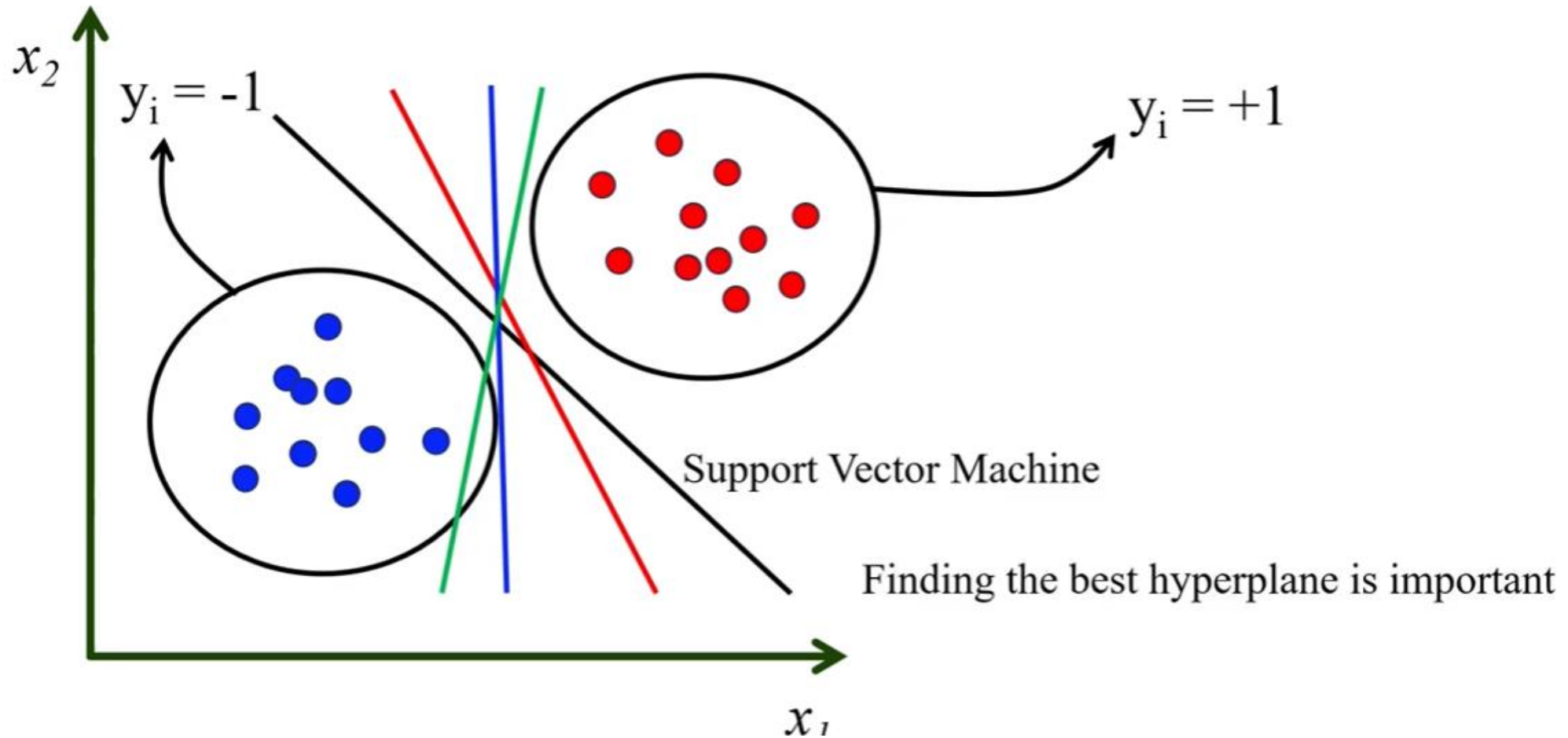
# Support Vector Machine



# Support Vector Machine

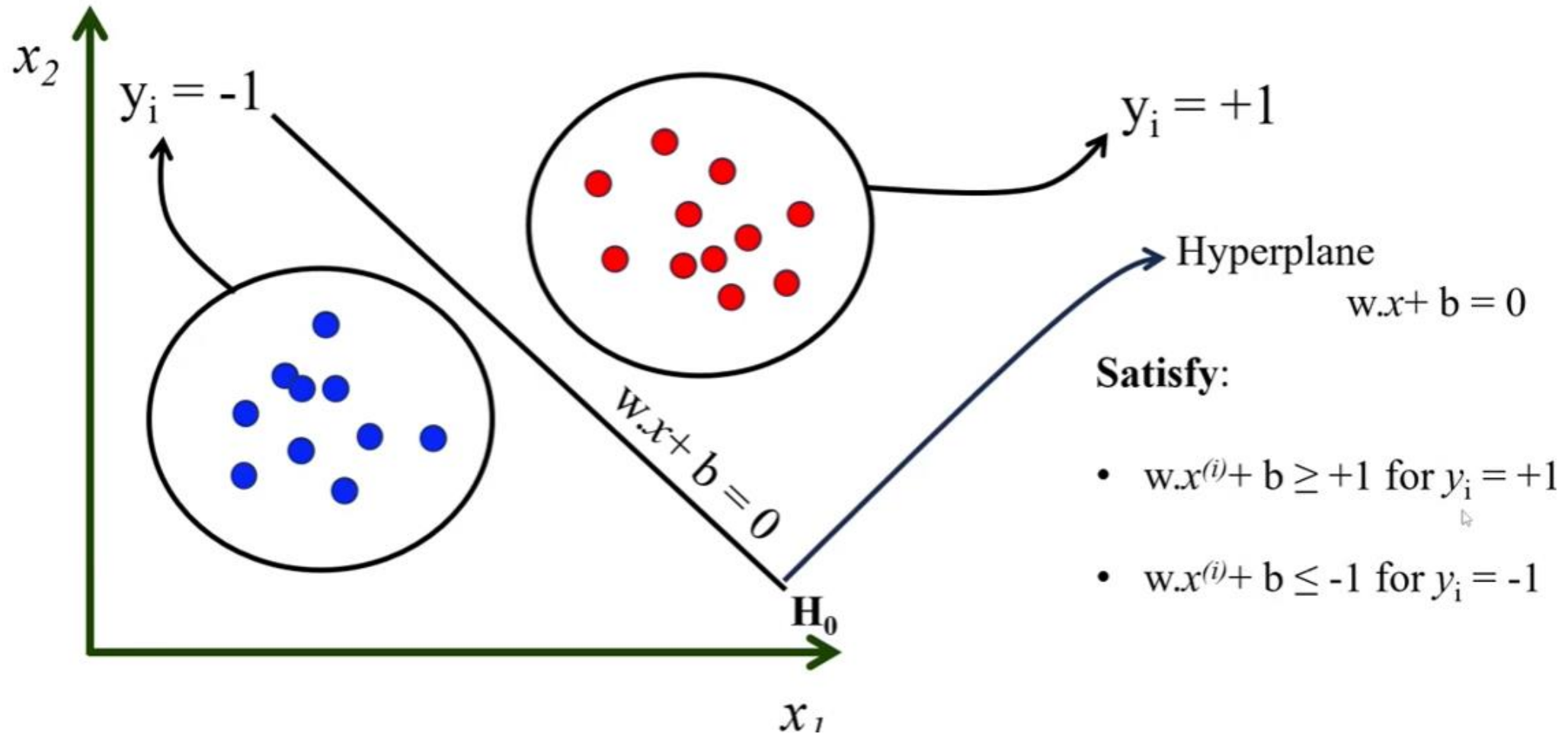


# Support Vector Machine

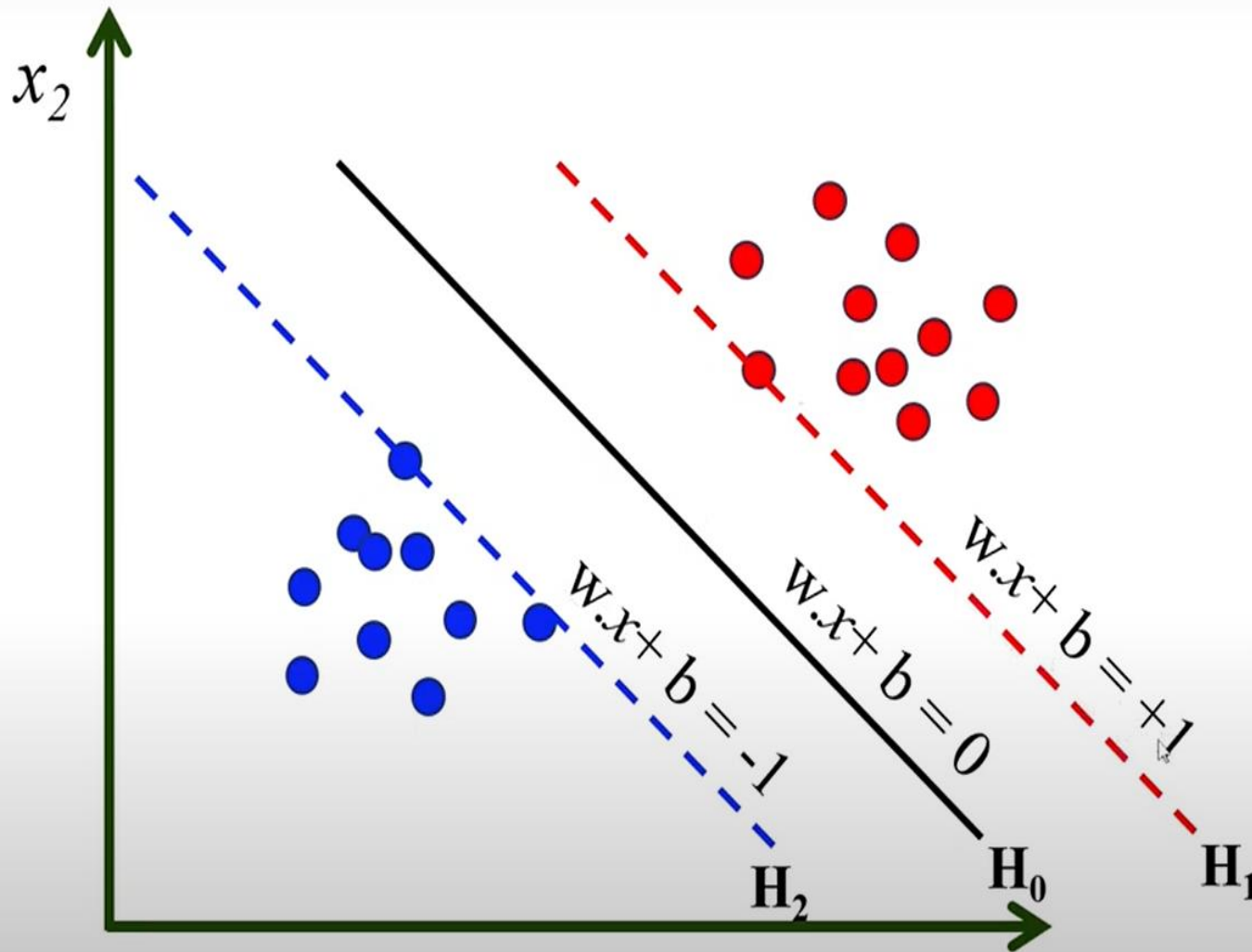




# Support Vector Machine



# Support Vector Machine



## Hyperplane

$$w.x + b = 0$$

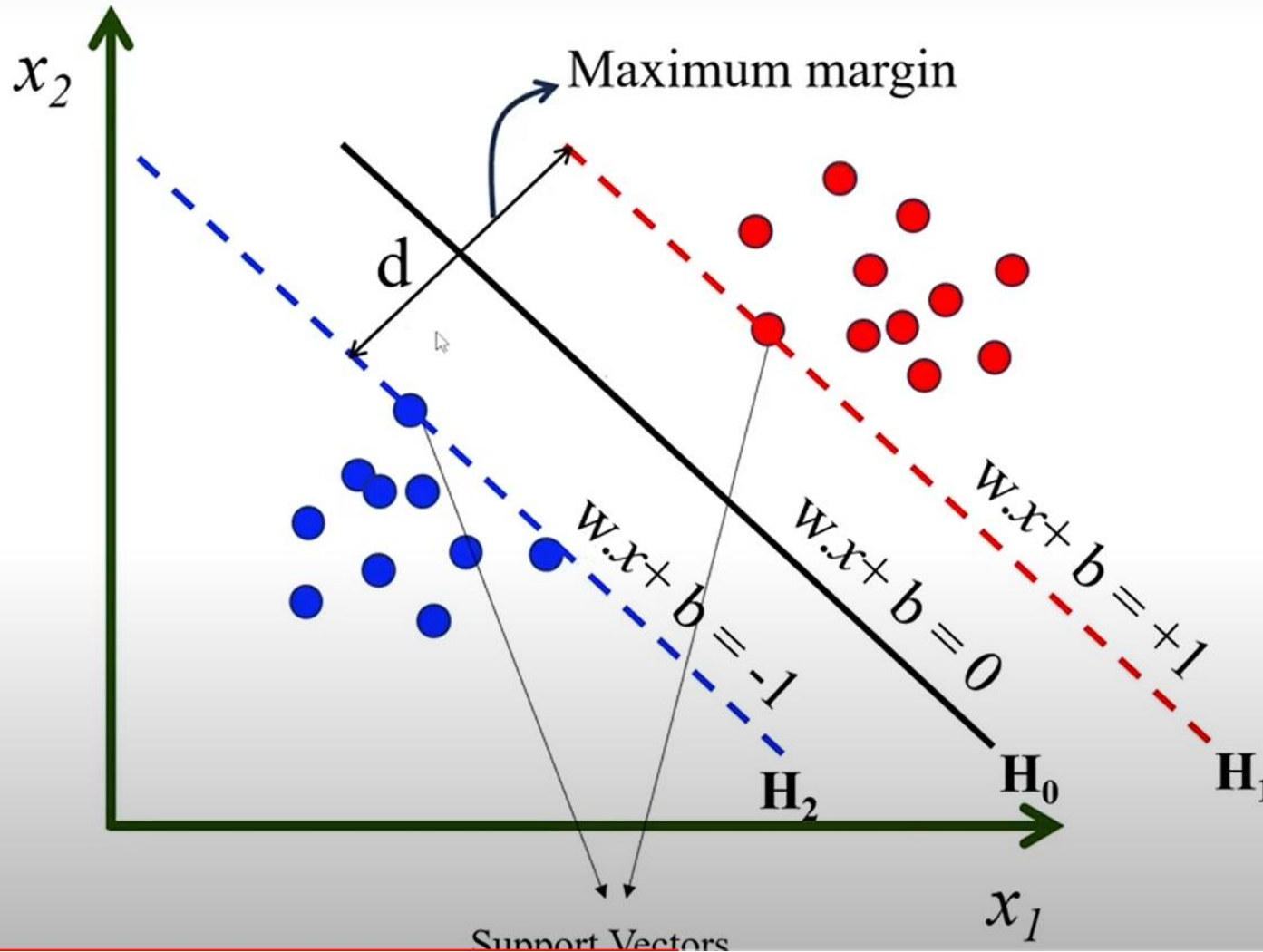
Satisfy:

- $w.x^{(i)} + b \geq +1$  for  $y_i = +1$
- $w.x^{(i)} + b \leq -1$  for  $y_i = -1$

$x = (x_1, x_2)$  is data point

$w = (w_1, w_2)$  is weights of equation

# Support Vector Machine



## Hyperplane

$$w.x + b = 0$$

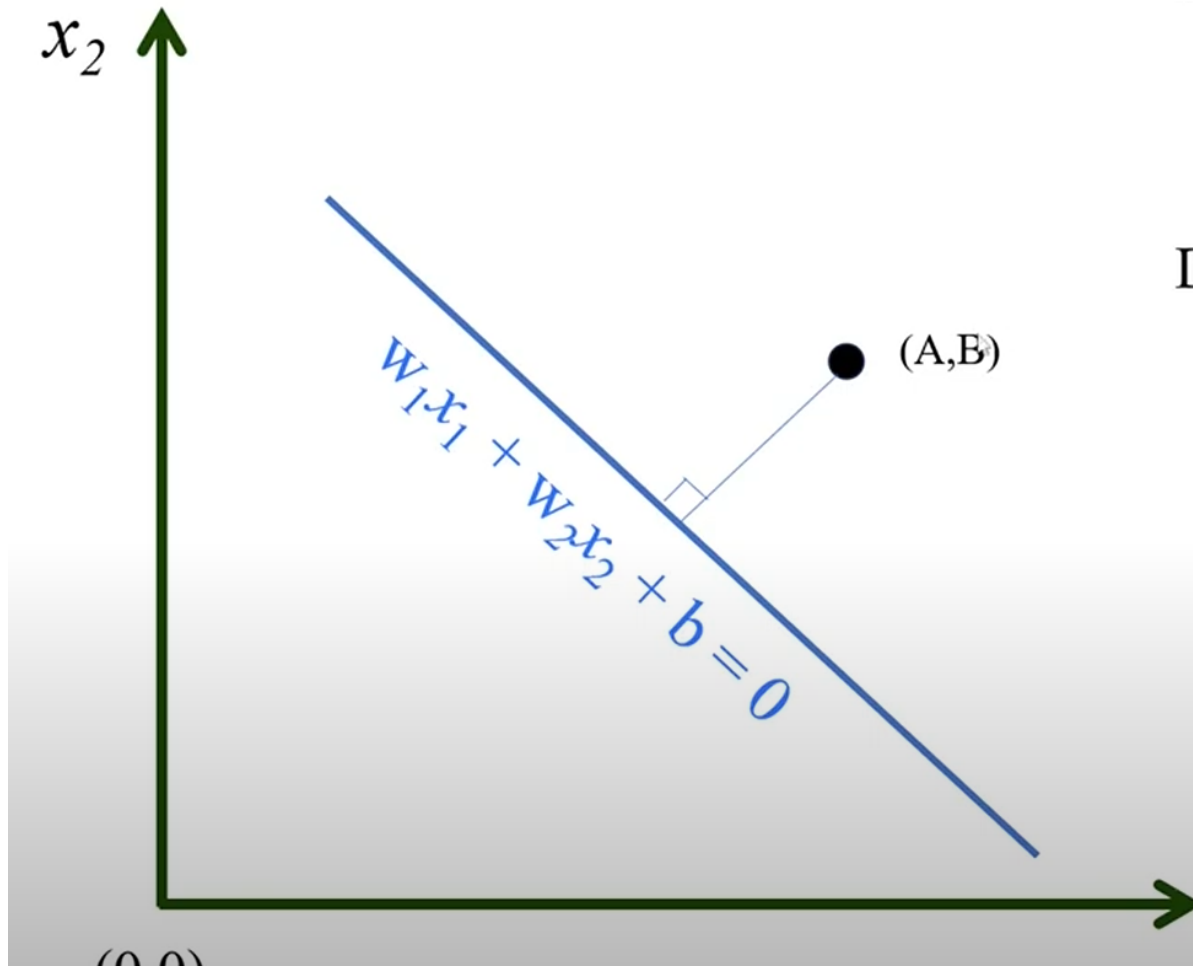
## Satisfy:

- $w.x^{(i)} + b \geq +1$  for  $y_i = +1$
- $w.x^{(i)} + b \leq -1$  for  $y_i = -1$

$x = (x_1, x_2)$  is data point

$w = (w_1, w_2)$  is weights of equation

# Support Vector Machine



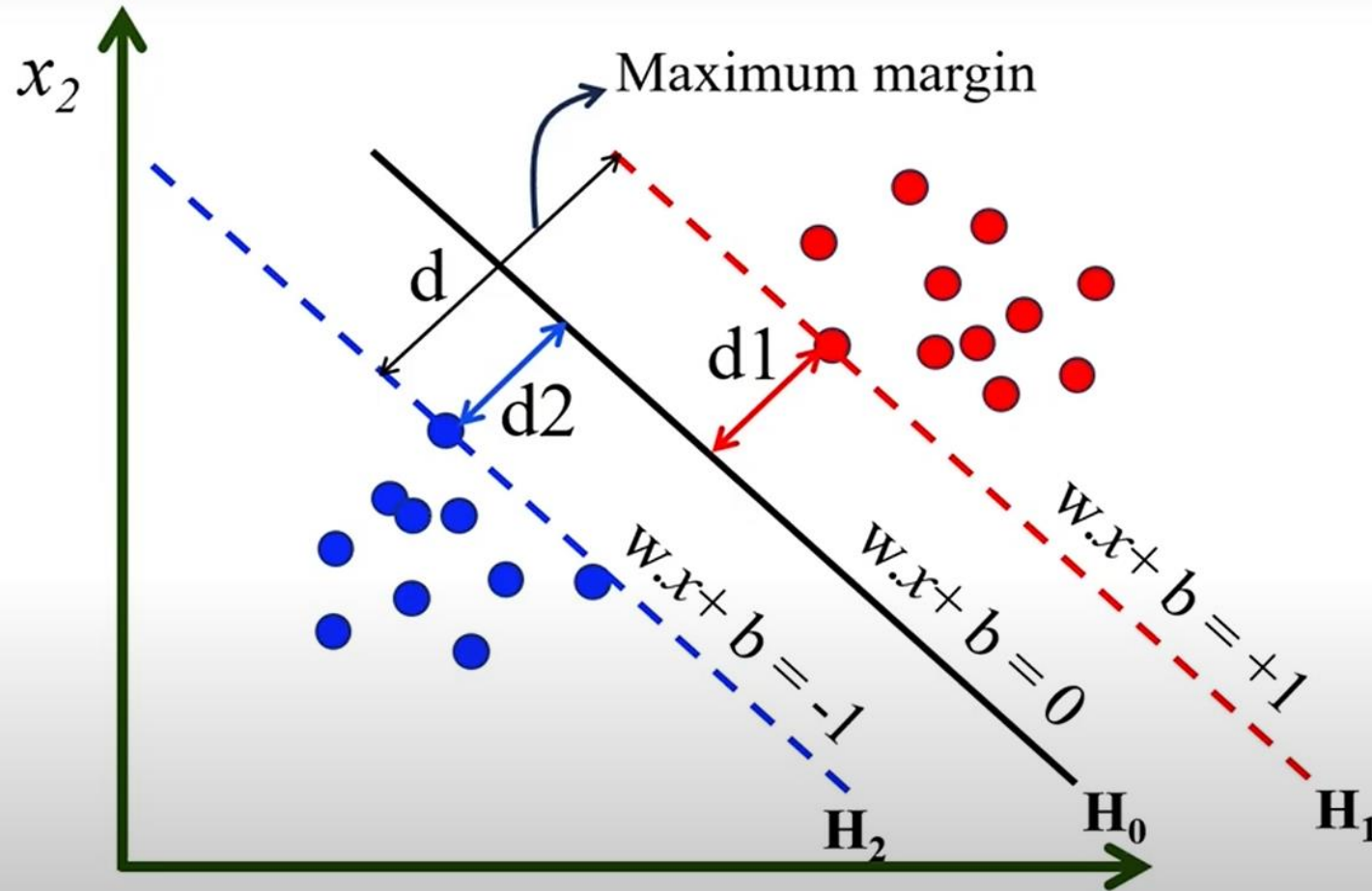
Linear Equation

$$w_1x_1 + w_2x_2 + b = 0$$

Distance from a point  $(A, B)$  to line:

$$\frac{|w_1A + w_2B + b|}{\sqrt{w_1^2 + w_2^2}} = \frac{|w_1A + w_2B + b|}{||w||}$$

# Support Vector Machine



## Hyperplane

$$w \cdot x + b = 0$$

$x = (x_1, x_2)$  is data point

$w = (w_1, w_2)$  is weights of equation

Distance between  $H_0$  and  $H_1$ :

$$d_1 = \frac{|w^T x + b|}{\|w\|} = \frac{1}{\|w\|} = d_2$$

$$\text{Maximum margin } d = \frac{2}{\|w\|}$$



Minimizing  $\|w\|$

# Support Vector Machine

**Generally:**

➤ With condition that there are no data points between  $H_1$  and  $H_2$

$$\left. \begin{array}{l} \bullet \mathbf{w}^T \mathbf{x}_i + b \geq +1 \text{ when } y_i = +1 \\ \bullet \mathbf{w}^T \mathbf{x}_i + b \leq -1 \text{ when } y_i = -1 \end{array} \right\} \mathbf{y}_i(\mathbf{w}^T \mathbf{x}_i + b) \geq +1$$

➤ Maximum margin  $d = \frac{2}{\|\mathbf{w}\|}$   Minimizing  $\|\mathbf{w}\|$

**Minimizing  $\|\mathbf{w}\|$  such that  $\mathbf{y}_i(\mathbf{w}^T \mathbf{x}_i + b) \geq +1$**

# Support Vector Machine

## Hinge Loss

Hinge Loss (L) of one data point:

$$L(y, f(x)) = \max(0, 1 - y * f(x))$$

- $y$  is the true class label (+1 or -1).
- $f(x)$  is the signed distance of the data point  $x$  to the decision boundary. It is essentially the inner product between the feature vector  $x$  and the weight vector  $w$ , plus the bias term  $b$

$$(f(x) = w * x + b).$$

- The loss function is non-negative, penalizing misclassified points ( $L \geq 0$ ).

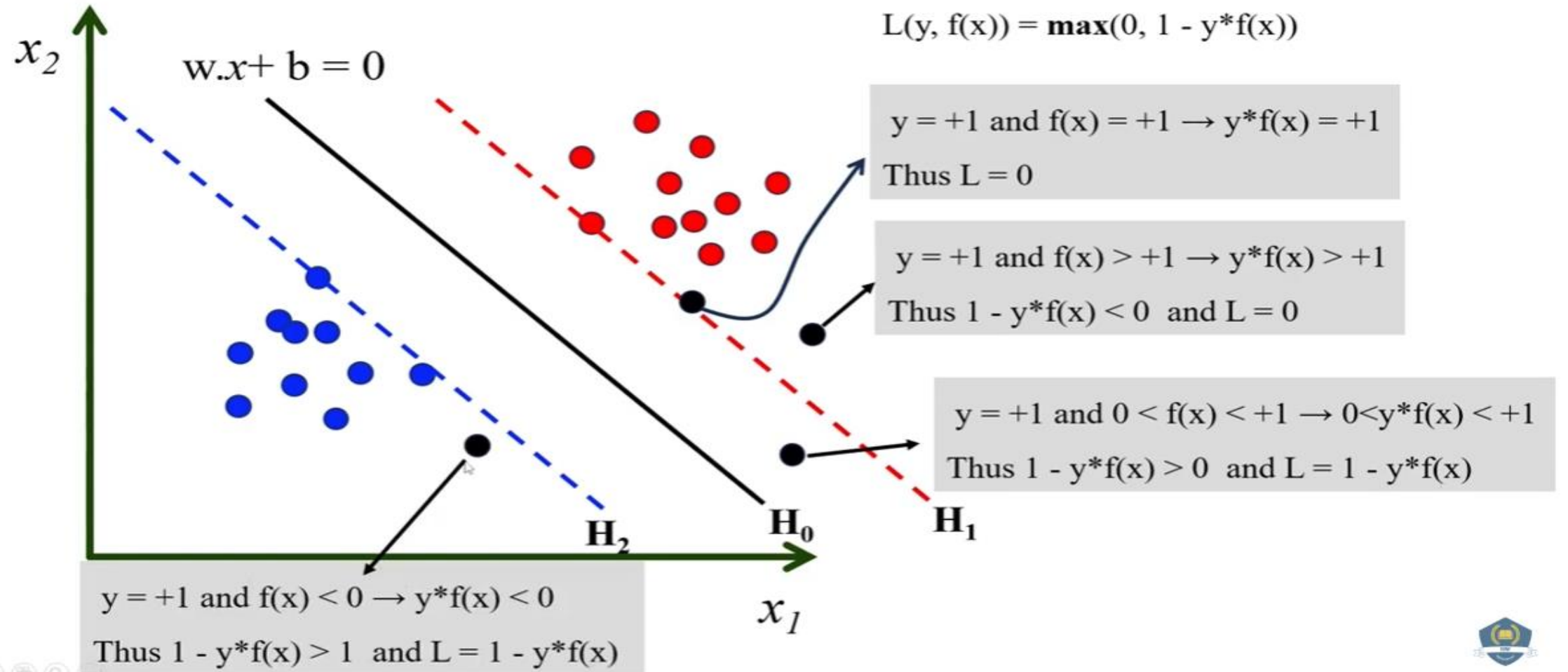
✓ If  $y * f(x) < 1$ , then  $1 - y * f(x) > 0$  and  $L = 1 - y * f(x)$

✓ If  $y * f(x) > 1$ , then  $1 - y * f(x) < 0$  and  $L = 0$

✓ If  $y * f(x) = 1$ , then  $1 - y * f(x) = 0$  and  $L = 0$

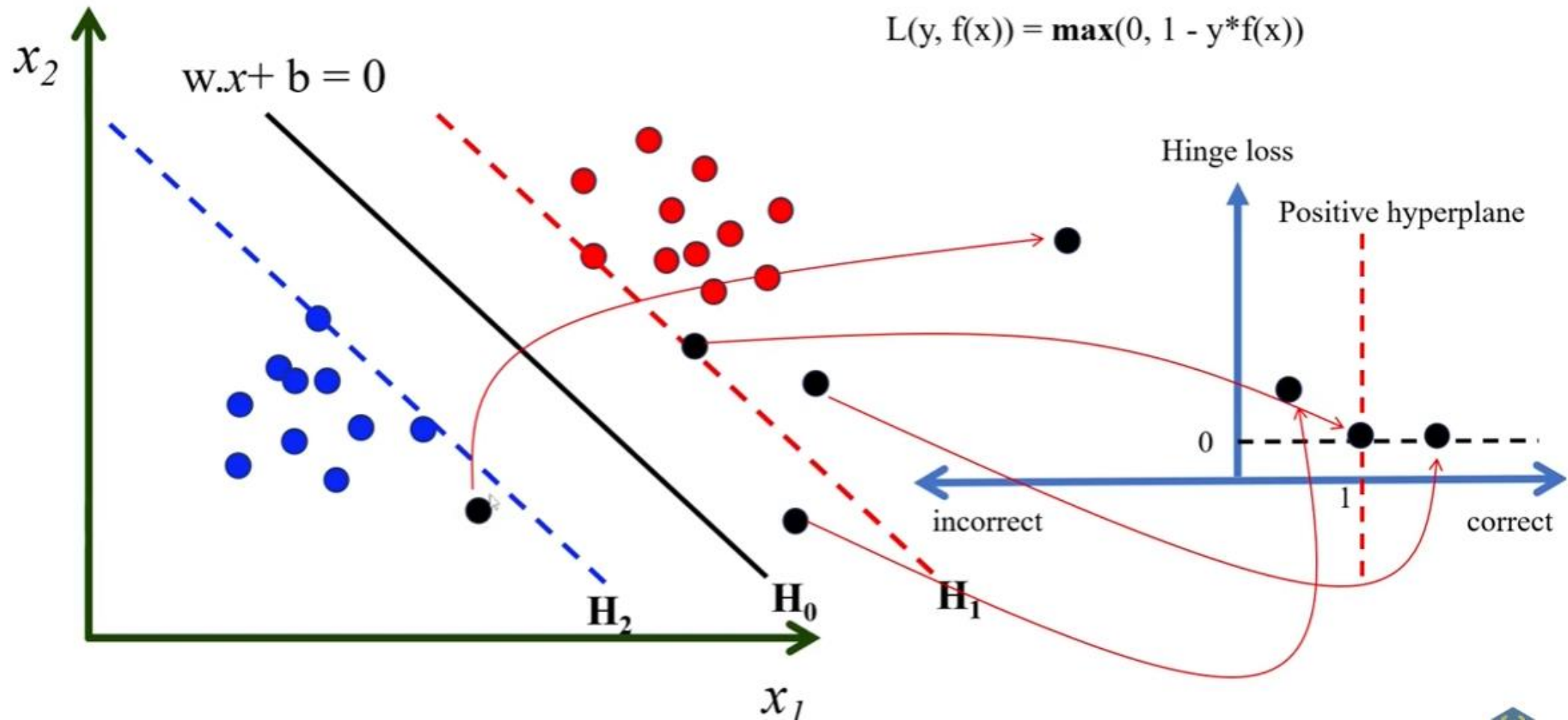


# Support Vector Machine





# Support Vector Machine



# Support Vector Machine

The objective of SVM is to **minimize the sum of hinge losses** over all data points, while also trying to **maximize the margin**

SVM will minimize

$$\frac{\|w\|^2}{2} + C \sum_i^N \max(0, 1 - y * f(x))$$

C is a regularization parameter that controls the trade-off between **maximizing margin** and **minimizing the hinge loss** (L)

- Large C → SVM emphasizes on minimizing hinge loss (L)
- Small C → SVM emphasizes on maximizing margin